

Table des matières

Liste des figures	v
Liste des tableaux	vi
Introduction générale	1
Contexte	2
Problématique	2
Objectifs	3
Organisation du rapport	3
1 Contexte et concepts fondamentaux	5
1.1 Introduction	5
1.2 Les attaques par déni de service distribué (DDoS)	5
1.2.1 Définition	5
1.2.2 Architecture d'une attaque DDoS	6
1.2.3 Classification des attaques DDoS	6
1.2.3.1 Attaques volumétriques	6
1.2.3.2 Attaques sur les protocoles réseau	7
1.2.3.3 Attaques applicatives (couche 7)	7
1.2.4 Impact et données statistiques	8
1.3 Infrastructure de mitigation DDoS	8
1.3.1 Les <i>scrubbing centers</i> : définition et rôle	8
1.3.2 Les réseaux de diffusion de contenu (CDN) comme bouclier anti-DDoS	9
1.3.3 Les pare-feu et systèmes de prévention d'intrusion (IPS)	10
1.3.3.1 Pare-feu traditionnels et nouvelle génération (NGFW)	10
1.3.3.2 Systèmes de prévention d'intrusion (IPS)	10
1.3.4 Le <i>blackholing</i> et le RTBH (<i>Remote Triggered Black Hole</i>)	11
1.3.5 Les services de mitigation cloud (<i>Cloud-based DDoS Protection</i>)	12
1.3.6 Techniques de redirection du trafic	13
1.3.7 Synthèse et comparaison des infrastructures de mitigation	14

1.3.8	Limites des approches centralisées	14
1.4	Architecture pair-à-pair pour la coordination inter-nœuds	15
1.4.1	Définition et pertinence du modèle pair-à-pair	15
1.4.2	Topologies et protocoles P2P applicables à la mitigation	16
1.4.3	Défis du P2P appliqué à la sécurité réseau	17
1.5	Conclusion	17
2	État de l'art	19
2.1	Introduction	19
2.2	DOTS — DDoS Open Threat Signaling	19
2.2.1	Contexte et motivation	19
2.2.2	Architecture et fonctionnement	19
2.2.2.1	Canal de signalisation (<i>Signal Channel</i>)	20
2.2.2.2	Canal de données (<i>Data Channel</i>)	21
2.2.3	Limites identifiées	21
2.3	DDoS Clearing House — Partage collaboratif de <i>fingerprints</i>	22
2.3.1	Contexte et motivation	22
2.3.2	Architecture et fonctionnement	22
2.3.2.1	Le <i>DDoS Fingerprint</i> : structure et contenu	22
2.3.2.2	Architecture logicielle	22
2.3.3	Limites identifiées	23
2.4	NaWas / NBIP — Centre national de <i>scrubbing</i> mutualisé	24
2.4.1	Contexte et architecture	24
2.4.1.1	Mécanisme de redirection BGP	24
2.4.2	Limites identifiées	25
2.5	DefCOM — <i>Distributed Cooperative Overlay Defense</i>	26
2.5.1	Contexte et architecture	26
2.5.1.1	Topologie et rôles des nœuds	26
2.5.1.2	Mécanisme de <i>Rate Limiting</i> coopératif	27
2.5.1.3	Protocole de communication overlay	27
2.5.2	Limites identifiées	28
2.6	Synthèse comparative	29
2.7	Conclusion	30
3	Conception et Implémentation de ShieldNet	31
3.1	Introduction	31
3.2	Architecture Générale de ShieldNet	32
3.2.1	Vue d'ensemble	32
3.2.2	Pile technologique	32
3.2.3	Structure du projet	32

3.3	Modèle de Données Relationnel	33
3.3.1	Vue d'ensemble du schéma	33
3.3.2	Table PEERS	34
3.3.3	Table HELP_SESSIONS	35
3.3.4	Initialisation et synchronisation de la base	35
3.4	Module d'Authentification : JWT RS256 et mTLS	36
3.4.1	Principes de sécurité	36
3.4.2	Architecture JWT asymétrique	36
3.4.3	Génération du token	36
3.4.4	Vérification du token (middleware)	37
3.4.4.1	Gestion des routes publiques	37
3.4.5	Correction d'un bug critique de bootstrapping	38
3.4.6	Middleware mTLS	38
3.5	Module de Découverte et Gestion des Pairs	39
3.5.1	Vue d'ensemble	39
3.5.2	Enregistrement transactionnel des pairs	39
3.5.3	Protocole de heartbeat	40
3.5.4	Machine à états des pairs	41
3.6	Module de Coalition Anti-DDoS	41
3.6.1	Cycle de vie d'une attaque	41
3.6.2	Détection et alerte	42
3.6.3	Négociation et redirection	42
3.6.4	Clôture et mise à jour des crédits	42
3.7	Algorithme WSM de Sélection des Pairs	43
3.7.1	Formalisation mathématique	43
3.7.2	Poids AHP validés	44
3.7.3	Implémentation	44
3.7.4	Contrainte d'allocation	45
3.7.5	Exemple numérique	45
3.8	Modèle de Confiance PeerTrust	46
3.8.1	Formulation mathématique	46
3.8.2	Poids contextuels de sévérité	46
3.8.3	Niveaux de confiance	47
3.8.4	Implémentation du calcul	47
3.8.5	Persistance et actions automatiques	48
3.8.6	Exemple de dégradation du score	49
3.8.7	API de gestion de la confiance	49
3.9	Service de Heartbeat Automatique	49
3.9.1	Architecture du service	49

3.9.2	Gestion des pairs injoignables	50
3.10	Infrastructure de Déploiement	51
3.10.1	Conteneurisation Docker	51
3.10.2	Génération des certificats TLS	52
3.10.3	Client HTTPS/mTLS	52
3.11	Script de Démonstration	53
3.11.1	Architecture du scénario	53
3.11.2	Phase 0 — Réinitialisation	53
3.11.3	Phase 3 — Simulation de l’attaque	54
3.11.4	Phase 4 — Résultat de la sélection WSM	54
3.11.5	Résultats de la démonstration	54
3.12	Monitoring et Journalisation	55
3.12.1	Journal d’audit	55
3.12.2	Endpoint de monitoring	55
3.13	Sécurité et Considérations de Production	56
3.13.1	Mesures de sécurité implémentées	56
3.13.2	Limitations actuelles et perspectives	56
3.14	Synthèse et Bilan de la Contribution	56
	Bibliographie	58

Table des figures

1.1	Architecture générale d’une attaque DDoS.	6
1.2	Processus de mitigation via un <i>scrubbing center</i>	9
1.3	Principe de protection anti-DDoS par un CDN avec Anycast.	10
1.4	Mécanisme de RTBH via BGP : la victime annonce un préfixe avec une communauté RTBH, et le FAI redirige le trafic correspondant vers null0.	12
2.1	Architecture de référence du protocole DDoS Open Threat Signaling (DOTS) : séparation entre le canal de signalisation (CoAP/DTLS/UDP) et le canal de données (RESTCONF/HTTPS/TCP).	20
2.2	Architecture du <i>DDoS Clearing House</i> : génération d’empreintes DDoS par le composant Dissector, stockage dans la base DDoS DB, puis utilisation par l’équipe opérationnelle pour générer des règles de mitigation.	23
2.3	Architecture simplifiée du service NaWas : redirection du trafic vers un centre de scrubbing mutualisé via annonce BGP et analyse des flux avant application des mécanismes de mitigation.	25
2.4	Arbre de limitation de débit coopératif dans DefCOM : propagation de la demande de la victime vers les sources.	28
3.1	Architecture distribuée de ShieldNet — 4 nœuds Docker interconnectés	32
3.2	Flux d’authentification JWT RS256 dans ShieldNet	36
3.3	Machine à états du statut d’un pair	41
3.4	Cycle de vie d’une session de coopération DDoS	41

Liste des tableaux

1.1	Synthèse comparative des catégories d’attaques DDoS.	8
1.2	Comparaison des principaux services de mitigation cloud.	12
1.3	Comparaison des infrastructures de mitigation DDoS.	14
2.1	Comparaison des systèmes de défense collaborative contre les attaques DDoS.	29
3.1	Pile technologique de ShieldNet	32
3.2	Tables de la base de données ShieldNet	34
3.3	Champs clés de la table HELP_SESSIONS	35
3.4	Endpoints du module de découverte	39
3.5	Poids AHP pour l’algorithme WSM	44
3.6	Exemple de calcul WSM — sélection pour 25 Gbps de trafic excédentaire	46
3.7	Poids contextuels selon la sévérité de l’attaque	46
3.8	Niveaux de confiance et seuils associés	47
3.9	Endpoints du module Trust	49
3.10	Configuration des 4 nœuds Docker	51
3.11	Les 7 phases de la démonstration ShieldNet	53
3.12	Résultats de la démonstration ShieldNet	55
3.13	Types d’événements d’audit enregistrés	55
3.14	Limitations identifiées et améliorations envisagées	56
3.15	Bilan des contributions implémentées	57

Introduction générale

La transformation numérique massive des organisations, l’essor du cloud et de l’Internet des objets (Internet of Things (IoT)) ont profondément renforcé la dépendance aux infrastructures réseau et aux systèmes d’information. La disponibilité continue des services numériques est devenue une exigence fondamentale, tant pour les acteurs économiques que pour les institutions et les utilisateurs. Dans cet environnement fortement interconnecté, toute interruption de service peut engendrer des conséquences opérationnelles, financières et réputationnelles significatives.

Les attaques par déni de service distribué (Distributed Denial of Service (DDoS)) constituent aujourd’hui l’une des principales menaces affectant la disponibilité des services numériques. Ces attaques consistent à générer un volume massif de trafic malveillant, distribué à partir de multiples sources, dans le but de saturer les ressources d’une cible et d’entraîner une indisponibilité partielle ou totale du service. Leur simplicité conceptuelle contraste avec leur efficacité opérationnelle, notamment lorsqu’elles exploitent des infrastructures distribuées à grande échelle et des botnets composés de milliers, voire de millions de dispositifs compromis.

Selon le rapport Cloudflare sur les menaces DDoS au premier trimestre 2025, 20,5 millions d’attaques ont été identifiées, soit une augmentation de 358 % par rapport à l’année précédente [2]. Certains incidents récents ont atteint des volumes dépassant plusieurs téraoctets par seconde, illustrant la capacité des attaquants à mobiliser des botnets massifs et des infrastructures distribuées à grande échelle.

Les secteurs les plus ciblés incluent notamment la finance, les télécommunications, les services en ligne à forte disponibilité, ainsi que les infrastructures publiques et les plateformes de commerce électronique. Les attaques multi-vecteurs, combinant saturation volumétrique, exploitation de protocoles réseau et ciblage applicatif (couche 7), représentent désormais une part significative des incidents observés, rendant les mécanismes de défense traditionnels plus difficiles à maintenir efficaces.

Les solutions actuelles reposent majoritairement sur des architectures centralisées, telles que les *scrubbing centers* ou les services de mitigation cloud. Bien que ces solutions offrent une capacité de traitement élevée, elles présentent des limites structurelles : dépendance à un point unique de décision, latence due à la redirection du trafic, coûts élevés et absence de coordination dynamique entre acteurs indépendants.

Face à ces limites, une nouvelle approche émerge : la **défense collaborative distribuée**. Plutôt que de traiter les attaques de manière isolée, plusieurs entités autonomes peuvent coopérer afin de partager des informations, synchroniser leurs actions et renforcer leur résilience collective. C'est dans ce cadre que s'inscrit ce projet de fin d'études, visant à concevoir une infrastructure de coordination inter-nœuds pour la défense collaborative contre les attaques DDoS.

Contexte

Les solutions classiques de mitigation DDoS reposent majoritairement sur des architectures centralisées, telles que des centres de nettoyage du trafic (*scrubbing centers*) ou des services cloud spécialisés. Ces approches permettent d'absorber de forts volumes de trafic malveillant et d'appliquer des mécanismes de filtrage avancés. Toutefois, elles présentent plusieurs limites structurelles.

D'une part, la centralisation introduit des points de concentration critiques, susceptibles de devenir eux-mêmes des goulots d'étranglement ou des cibles d'attaque. D'autre part, les coûts associés à ces services peuvent être significatifs, ce qui limite leur accessibilité pour certaines organisations, notamment les PME et les collectivités. Enfin, la latence induite par la redirection du trafic vers des infrastructures distantes peut impacter la qualité de service.

Parallèlement, l'évolution des menaces complexifie la détection et la coordination des réponses. Les attaques modernes exploitent des botnets massifs, des vulnérabilités applicatives et des techniques d'évasion sophistiquées. Elles peuvent évoluer dynamiquement en fonction des contre-mesures mises en place. Face à ces attaques adaptatives, une simple réaction locale et isolée montre rapidement ses limites.

Dans ce cadre, l'idée d'une défense collaborative et distribuée — reposant sur la coordination entre plusieurs nœuds de protection capables d'échanger des informations en temps réel et de mutualiser leurs ressources — apparaît comme une piste pertinente et innovante.

Problématique

Les attaques DDoS atteignent aujourd'hui des volumes et une complexité qui dépassent les capacités de mitigation d'infrastructures isolées. Face à cette menace, les organisations utilisent principalement des solutions centralisées : *scrubbing centers* ou services cloud de mitigation.

Ces architectures présentent plusieurs limitations techniques :

- dépendance aux prestataires externes ;
- coûts élevés pour les PME et collectivités ;

- latence due à la redirection du trafic ;
- points uniques de défaillance (Single Point of Failure (SPOF)) ;
- absence de mutualisation des ressources de défense.

De plus, la défense reste individuelle. Les mécanismes actuels se limitent au partage d'informations, sans mutualisation opérationnelle des capacités de mitigation ni coordination des contre-mesures. En cas d'attaque distribuée touchant plusieurs cibles simultanément, chaque organisation réagit de son côté, ce qui réduit l'efficacité globale.

Objectifs

L'objectif principal de ce projet est de concevoir et d'implémenter un mécanisme de coordination inter-nœuds destiné à renforcer la défense collaborative contre les attaques DDoS.

Pour atteindre cet objectif, le travail consiste à :

1. Réaliser une étude approfondie des attaques DDoS, des mécanismes de mitigation existants et des approches collaboratives en environnement distribué.
2. Analyser les limites des solutions centralisées actuelles et identifier les besoins en matière de coordination et de partage de ressources entre nœuds autonomes.
3. Définir les principaux cas d'usage de coordination entre nœuds de protection, notamment l'alerte, l'entraide et la redirection de trafic.
4. Concevoir un protocole distribué d'échange inter-nœuds précisant les règles de communication et de prise de décision.
5. Développer un prototype permettant l'échange sécurisé de métadonnées d'attaque et la synchronisation des actions de mitigation entre plusieurs nœuds.
6. Mettre en place un environnement de test simulant un réseau de nœuds co-opérants afin d'évaluer la robustesse, la latence et la cohérence des mécanismes proposés.
7. Réaliser une documentation technique détaillant l'architecture et le fonctionnement du système développé.

Organisation du rapport

Le présent mémoire est structuré comme suit :

- **Chapitre 1 : Contexte et concepts fondamentaux.** Ce chapitre présente les fondements théoriques des attaques DDoS, la taxonomie des vecteurs d'attaque et les mécanismes de mitigation existants (*scrubbing centers*, techniques de redirection, pipeline de filtrage), ainsi que les principes des architectures pair-à-pair applicables à la coordination inter-nœuds.

- **Chapitre 2 : État de l’art.** Ce chapitre propose un état de l’art des approches collaboratives et distribuées en matière de défense contre les DDoS. Cinq systèmes représentatifs sont analysés en profondeur : DOTS, DDoS Clearing House, NaWas/NBIP et DefCOM. Une synthèse comparative identifie les lacunes motivant notre contribution.
- **Chapitre 3 : Modélisation du système.** Ce chapitre décrit le modèle du système et formalise les hypothèses retenues, les cas d’usage de coordination et les exigences fonctionnelles et non fonctionnelles.
- **Chapitre 4 : Conception.** Ce chapitre détaille la conception du protocole distribué de coordination inter-nœuds et de l’architecture d’API sécurisée pour l’échange de métadonnées d’attaque.
- **Chapitre 5 : Implémentation.** Ce chapitre présente l’implémentation du prototype et l’environnement expérimental mis en place pour la validation.
- **Chapitre 6 : Résultats et évaluation.** Ce chapitre expose les résultats expérimentaux obtenus et leur analyse en termes de temps de réponse, de passage à l’échelle et de résilience.

Enfin, une **conclusion générale** synthétise les contributions du travail et propose des perspectives d’évolution.

Chapitre 1

Contexte et concepts fondamentaux

1.1 Introduction

La sécurité des infrastructures réseau est aujourd’hui l’un des enjeux majeurs de l’informatique moderne. Parmi les menaces les plus dévastatrices, les attaques par déni de service distribué (DDoS) occupent une place prépondérante, en raison de leur capacité à paralyser des services critiques en saturant les ressources d’un ou plusieurs systèmes cibles.

Face à l’augmentation en volume et en sophistication de ces attaques, les approches de mitigation centralisées montrent rapidement leurs limites. Il devient alors indispensable d’envisager des mécanismes de défense collaborative et distribuée, où plusieurs nœuds de protection — notamment des *scrubbing centers* — coordonnent leurs actions en temps réel pour détecter, analyser et contrer ces menaces de manière collective.

Ce chapitre a pour objectif de poser les bases conceptuelles nécessaires à la compréhension du travail présenté dans ce mémoire. Nous abordons successivement la nature et la classification des attaques DDoS, les mécanismes d’infrastructure de mitigation existants, puis les principes des systèmes distribués appliqués à la coordination inter-nœuds.

1.2 Les attaques par déni de service distribué (DDoS)

1.2.1 Définition

Une attaque par déni de service (Denial of Service (DoS)) désigne toute action malveillante visant à rendre un service informatique indisponible pour ses utilisateurs légitimes, en épuisant les ressources du système cible (bande passante, mémoire, CPU, connexions). Lorsque cette attaque est menée simultanément depuis un grand nombre de machines distribuées, on parle d’attaque DDoS [12].

Mirkovic et Reiher définissent une attaque DDoS comme : « une tentative explicite

d'un attaquant de nier aux utilisateurs légitimes l'utilisation d'un service réseau, en submergeant la cible ou son infrastructure intermédiaire avec du trafic illégitime » [13].

1.2.2 Architecture d'une attaque DDoS

Une attaque DDoS repose sur une architecture à plusieurs niveaux impliquant différents acteurs (voir Figure 1.1) :

- **L'attaquant (*handler*)** : l'entité malveillante qui orchestre l'attaque depuis une machine maîtresse.
- **Le réseau de commande et contrôle (C&C)** : infrastructure permettant à l'attaquant de coordonner les agents d'attaque à distance, souvent dissimulée via des protocoles légitimes (IRC, HTTP, Pair-à-Pair (P2P)).
- **Les agents (*zombies* / *bots*)** : machines compromises (ordinateurs, serveurs, objets connectés IoT) qui exécutent les instructions reçues du C&C et génèrent le trafic malveillant.
- **La cible (*victim*)** : le système dont on cherche à épuiser les ressources — serveur web, infrastructure DNS, routeurs de transit, etc.

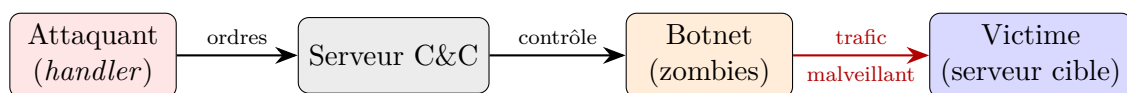


FIGURE 1.1 – Architecture générale d'une attaque DDoS.

Specht et Lee [23] soulignent que la compromission de milliers voire de millions de machines forme des botnets dont la puissance agrégée dépasse largement la capacité de réponse d'un seul nœud de défense, justifiant ainsi l'intérêt d'une défense distribuée et collaborative.

1.2.3 Classification des attaques DDoS

La littérature propose plusieurs taxonomies des attaques DDoS [12], [27]. Nous adoptons ici la classification en trois grandes catégories, largement répandue dans les travaux académiques et industriels. Le Tableau 1.1 en offre une synthèse comparative.

1.2.3.1 Attaques volumétriques

Ces attaques visent à saturer la bande passante du lien réseau de la victime ou de son Fournisseur d'Accès Internet (FAI). Elles constituent la catégorie la plus répandue et la plus facile à générer avec un botnet. On distingue notamment :

- **UDP Flood** : envoi massif de paquets UDP vers des ports aléatoires de la cible. L'effet principal réside dans la saturation de la bande passante par le volume brut du trafic entrant [12].

- **ICMP Flood (*Ping Flood*)** : saturation par des requêtes ICMP Echo Request en volume.
- **Attaques par amplification/réflexion** : exploitation de serveurs tiers (DNS, NTP, memcached) pour amplifier le volume du trafic. L'attaquant envoie des requêtes à adresse source *spoofée* (celle de la victime) vers des serveurs ouverts, qui répondent avec des paquets de taille bien supérieure. Selon Rossow [20], le facteur d'amplification DNS peut atteindre $28\times$ à $54\times$, et celui de NTP jusqu'à $556\times$. Les attaques via memcached ont démontré des facteurs dépassant $50\,000\times$ [1].

1.2.3.2 Attaques sur les protocoles réseau

Ces attaques exploitent des faiblesses dans les protocoles de communication (TCP, ICMP) pour épuiser les ressources de traitement des équipements réseau intermédiaires.

- **SYN Flood** : exploitation du mécanisme de *handshake* TCP en trois voies. L'attaquant envoie des paquets SYN avec adresses source *spoofées*, forçant le serveur à maintenir en mémoire des connexions semi-ouvertes jusqu'à épuisement de la table d'état (*backlog*). Eddy [4] décrit en détail les vulnérabilités du protocole TCP exploitées par ce type d'attaque.
- **Smurf Attack** : envoi de requêtes ICMP *broadcast* avec adresse source *spoofée* (celle de la victime), générant une avalanche de réponses vers la cible.

Note historique : Le *Ping of Death*, qui consistait à envoyer des paquets ICMP de taille supérieure au maximum autorisé, a constitué une menace dans les années 1990 mais est aujourd'hui corrigé.

1.2.3.3 Attaques applicatives (couche 7)

Ces attaques ciblent la couche application du modèle OSI et sont particulièrement difficiles à distinguer du trafic légitime, car elles utilisent des requêtes HTTP, HTTPS, DNS valides du point de vue syntaxique [27].

- **HTTP Flood** : envoi massif de requêtes HTTP GET ou POST vers un serveur web, saturant sa capacité de traitement sans nécessiter un volume de trafic important.
- **Slowloris** : maintien d'un maximum de connexions HTTP ouvertes vers le serveur cible en envoyant des en-têtes partiels à faible débit, empêchant le serveur d'accepter de nouvelles connexions légitimes.
- **DNS Query Flood** : envoi massif de requêtes DNS valides pour épuiser les ressources des serveurs de noms.

TABLE 1.1 – Synthèse comparative des catégories d’attaques DDoS.

Caractéristique	Volumétrique	Protocole	Applicative (L7)
Couche OSI ciblée	3–4 (Réseau /Transport)	3–4 (Réseau /Transport)	7 (Application)
Objectif principal	Saturer la bande passante	Épuiser les tables d’état	Épuiser les ressources applicatives
Volume requis	Très élevé (Gbps–Tbps)	Modéré	Faible à modéré
Difficulté de détection	Faible (anomalie volumétrique)	Moyenne	Élevée (trafic légitime en apparence)
Exemples	UDP Flood, Amplification	SYN Flood, Smurf	HTTP Flood, Slowloris
Mitigation typique	Filtrage, <i>blackholing</i>	SYN cookies, <i>rate limiting</i>	Analyse comportementale, WAF

1.2.4 Impact et données statistiques

Les attaques DDoS engendrent des pertes économiques considérables. Selon le rapport NETSCOUT Threat Intelligence Report 2023, plus de 7,9 millions d’attaques DDoS ont été enregistrées sur la seule première moitié de l’année 2023, soit une moyenne de plus de 44 000 attaques par jour [16]. Zargar et al. [27] estiment que le coût moyen d’une attaque DDoS pour une entreprise peut dépasser 50 000 dollars par heure d’interruption de service, incluant les pertes commerciales directes, les coûts de mitigation et les dommages réputationnels.

1.3 Infrastructure de mitigation DDoS

La mitigation des attaques DDoS repose sur un ensemble d’infrastructures et de techniques déployées à différents niveaux du réseau. Cette section présente les principales solutions existantes, leurs mécanismes de fonctionnement et leurs limites respectives.

1.3.1 Les *scrubbing centers* : définition et rôle

Un *scrubbing center* (ou centre de nettoyage du trafic) est une infrastructure spécialisée dont la mission est d’analyser le trafic réseau entrant, de filtrer les paquets malveillants, puis de renvoyer uniquement le trafic légitime vers la destination finale. Ce mécanisme est souvent désigné sous le terme de « *clean pipe* » [9].

Le processus de mitigation via *scrubbing center* se déroule en plusieurs étapes distinctes (voir Figure 1.2) :

1. **Détection** : une anomalie dans le trafic déclenche une alerte (analyse Net-Flow/sFlow, seuils de bande passante, corrélation multi-sources).
2. **Redirection** : le trafic destiné à la victime est redirigé vers le *scrubbing center* via des mécanismes de routage (Border Gateway Protocol (BGP), DNS).
3. **Analyse et filtrage** : le centre analyse le trafic en appliquant un pipeline de traitement multi-couches (empreintes, comportement, limitation de débit).
4. **Réinjection** : le trafic nettoyé est retransmis à la victime via un tunnel Generic Routing Encapsulation (GRE) ou MPLS pour éviter les boucles de routage.

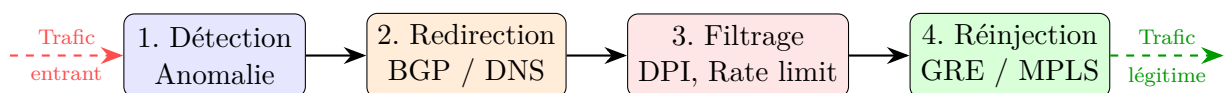


FIGURE 1.2 – Processus de mitigation via un *scrubbing center*.

Parmi les principaux fournisseurs de *scrubbing centers*, on trouve des acteurs comme Arbor Networks (Sightline/TMS), Akamai (Prolexic), Radware (DefensePro) et A10 Networks (Thunder TPS). Ces solutions offrent des capacités de traitement allant de quelques dizaines de Gbps à plusieurs Tbps selon l'infrastructure déployée.

1.3.2 Les réseaux de diffusion de contenu (CDN) comme bouclier anti-DDoS

Les **CDN** (*Content Delivery Networks*) tels que Cloudflare, Akamai et AWS CloudFront constituent une autre ligne de défense majeure contre les attaques DDoS. Leur principe repose sur la **distribution géographique** du contenu et du traitement du trafic à travers un réseau mondial de points de présence (PoP).

Le mécanisme de protection d'un CDN fonctionne comme suit :

1. **Interception par DNS** : les enregistrements DNS de la victime sont modifiés pour pointer vers les serveurs du CDN. Tout le trafic entrant transite donc par le réseau du CDN avant d'atteindre le serveur d'origine.
2. **Absorption volumétrique** : grâce à la technique **Anycast**, le même préfixe IP est annoncé depuis des centaines de PoP répartis mondialement. Le trafic est automatiquement dirigé vers le PoP le plus proche géographiquement, ce qui **distribue naturellement** la charge d'une attaque volumétrique sur l'ensemble du réseau.
3. **Filtrage en bordure** : chaque PoP applique des règles de filtrage (limitation de débit, WAF, challenges JavaScript/CAPTCHA) pour bloquer le trafic malveillant avant qu'il n'atteigne le serveur d'origine.

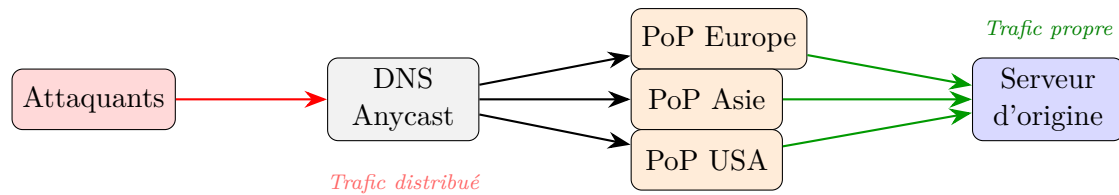


FIGURE 1.3 – Principe de protection anti-DDoS par un CDN avec Anycast.

4. **Transmission du trafic légitime** : seul le trafic validé est transmis au serveur d'origine via un canal sécurisé (HTTPS).

Cependant, les CDN présentent des limites :

- **Protection limitée à la couche applicative (HTTP/HTTPS)** : les CDN protègent principalement les services web. Les attaques ciblant d'autres protocoles (DNS autoritaire, VoIP, jeux en ligne, VPN) ne bénéficient pas de cette protection.
- **Dépendance à un fournisseur tiers** : l'ensemble du trafic transite par l'infrastructure du CDN, créant une dépendance forte et un risque de confidentialité.
- **Coûts variables** : la tarification au volume peut devenir prohibitive lors d'attaques massives prolongées.

1.3.3 Les pare-feu et systèmes de prévention d'intrusion (IPS)

Les **pare-feu** (*firewalls*) et les **systèmes de prévention d'intrusion** (*Intrusion Prevention Systems* — IPS) constituent la première ligne de défense déployée en périphérie du réseau de l'organisation.

1.3.3.1 Pare-feu traditionnels et nouvelle génération (NGFW)

Les pare-feu filtrent le trafic réseau en fonction de règles prédéfinies basées sur les adresses IP, les ports et les protocoles. Les **pare-feu de nouvelle génération** (NGFW — *Next-Generation Firewall*) ajoutent des capacités d'inspection applicative :

- **Inspection à états (*stateful inspection*)** : suivi des connexions TCP pour détecter les anomalies (SYN Flood, connexions incomplètes).
- **Inspection applicative (couche 7)** : analyse du contenu des paquets pour identifier les requêtes malveillantes (injections SQL, requêtes HTTP anormales).
- **Limitation de débit (*rate limiting*)** : restriction du nombre de connexions par source IP et par unité de temps.

1.3.3.2 Systèmes de prévention d'intrusion (IPS)

Les IPS analysent le trafic en temps réel pour détecter et bloquer les comportements malveillants en se basant sur :

- **Signatures** : correspondance avec une base de signatures d’attaques connues (similaire aux antivirus).
- **Anomalies** : détection de déviations par rapport à un profil de trafic normal (*baseline*).
- **Heuristiques** : analyse comportementale combinant plusieurs indicateurs pour détecter des attaques inconnues (*zero-day*).

Limites face aux attaques DDoS : les pare-feu et IPS sont dimensionnés pour le volume de trafic *normal* de l’organisation. Lors d’une attaque volumétrique, le pare-feu lui-même devient un goulot d’étranglement : sa capacité de traitement (mesurée en paquets par seconde — pps) est rapidement dépassée, ce qui peut provoquer l’effondrement de l’ensemble du périmètre de sécurité. De plus, les pare-feu à états consomment de la mémoire pour chaque connexion suivie, les rendant particulièrement vulnérables aux attaques d’épuisement de tables d’état (SYN Flood, TCP connection exhaustion).

1.3.4 Le *blackholing* et le RTBH (*Remote Triggered Black Hole*)

Le *blackholing* est une technique de dernier recours qui consiste à rediriger l’intégralité du trafic destiné à une adresse IP attaquée vers une route nulle (`null0` / *bit bucket*), supprimant ainsi tout le trafic — légitime et malveillant — vers cette destination.

Le mécanisme **RTBH** (*Remote Triggered Black Hole*, RFC 5635 [8]) permet d’automatiser ce processus via BGP :

1. Le réseau victime annonce le préfixe attaqué avec une **communauté BGP spéciale** (ex. : `AS:666`) à ses fournisseurs de transit.
2. Les routeurs des fournisseurs de transit, configurés pour reconnaître cette communauté, redirigent le trafic vers `null0`.
3. Le trafic d’attaque est ainsi éliminé *en amont*, avant qu’il ne sature les liens du réseau victime.

Avantage : le *blackholing* est extrêmement efficace pour protéger l’infrastructure réseau (les liens ne sont plus saturés).

Inconvénient majeur : il constitue une **victoire de l’attaquant**. En supprimant tout le trafic vers la cible — y compris le trafic légitime —, le *blackholing* produit exactement le résultat recherché par l’attaque : le déni de service. C’est pourquoi cette technique n’est utilisée qu’en dernier recours, lorsque le sacrifice d’un service est préférable à l’effondrement de l’infrastructure complète.

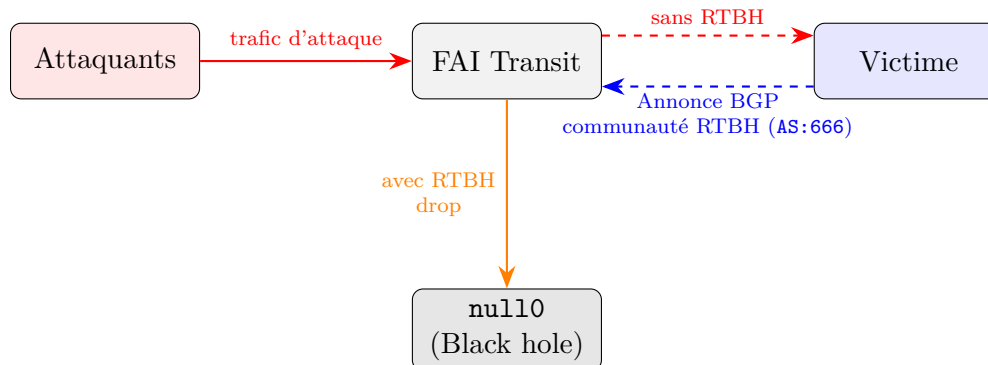


FIGURE 1.4 – Mécanisme de RTBH via BGP : la victime annonce un préfixe avec une communauté RTBH, et le FAI redirige le trafic correspondant vers `null0`.

1.3.5 Les services de mitigation cloud (*Cloud-based DDoS Protection*)

Les **services de mitigation cloud** sont proposés par des fournisseurs spécialisés (Cloudflare, AWS Shield, Azure DDoS Protection, Google Cloud Armor) qui disposent d’une infrastructure mondiale massive pour absorber les attaques DDoS.

Le fonctionnement repose sur le même principe que les *scrubbing centers*, mais avec deux différences majeures :

- **Capacité élastique** : les fournisseurs cloud disposent de capacités de traitement pouvant dépasser 100 Tbps en cumulé sur l’ensemble de leurs PoP, ce qui leur permet d’absorber les attaques les plus volumétriques.
- **Modèle *as-a-Service*** : la mitigation est proposée sous forme d’abonnement, évitant à l’organisation d’investir dans une infrastructure physique dédiée.

Le Tableau 1.2 compare les principaux services de mitigation cloud disponibles sur le marché.

TABLE 1.2 – Comparaison des principaux services de mitigation cloud.

Service	Capacité	Couches protégées	Activation
Cloudflare	348 Tbps (réseau global)	L3/L4 + L7 (WAF intégré)	Always-on (auto-matique)
AWS Shield Advanced	Élastique (infra AWS)	L3/L4 + L7 (avec WAF)	Always-on + sur demande
Azure DDoS Protection	Élastique (infra Azure)	L3/L4	Always-on
Google Cloud Armor	Élastique (infra Google)	L7 (HTTP/HTTPS)	Always-on
Akamai Prolexic	20+ Tbps (scrubbing dédié)	L3/L4 + L7	On-demand (BGP redirect)

Limites des services cloud :

- **Dépendance au fournisseur** : l'organisation confie l'intégralité de son trafic à un tiers, ce qui soulève des questions de confidentialité, de souveraineté des données et de *vendor lock-in*.
- **Coûts récurrents** : les abonnements premium (ex. : AWS Shield Advanced à 3 000 \$/mois + coûts de transfert de données) peuvent être prohibitifs pour les petites structures.
- **Latence additionnelle** : le reroutage du trafic via le réseau du fournisseur ajoute une latence (typiquement 5 à 30 ms selon la distance au PoP le plus proche).
- **Absence de coordination inter-fournisseurs** : chaque service opère de manière isolée. Il n'existe pas de mécanisme standardisé pour que Cloudflare, AWS et Akamai coordonnent leurs actions face à une attaque ciblant simultanément plusieurs de leurs clients.

1.3.6 Techniques de redirection du trafic

La redirection du trafic vers une infrastructure de mitigation (qu'il s'agisse d'un *scrubbing center*, d'un CDN ou d'un service cloud) est une opération critique. Plusieurs techniques sont utilisées en pratique :

- **BGP Blackholing et BGP Flowspec** : Le protocole BGP (RFC 4271 [18]) permet d'annoncer des préfixes IP pour rediriger le trafic. Le BGP Flowspec (RFC 5575 [10]) étend cette capacité en distribuant des règles de filtrage fin (correspondance sur les champs source/destination IP, port, protocole, taille de paquet, fragment) sous forme de Network Layer Reachability Information (NLRI) multi-champs, permettant aux routeurs de bordure d'appliquer des actions (`discard`, `rate-limit`, `redirect` vers VRF) sans intervention manuelle.
- **DNS Redirection** : Modification des enregistrements DNS de la victime pour pointer vers l'infrastructure de mitigation. Technique utilisée par les CDN (Cloudflare, Akamai) et les services cloud.
- **Anycast** : Déploiement du même préfixe IP sur plusieurs nœuds distribués géographiquement, permettant une absorption distribuée naturelle. Le trafic est dirigé vers le nœud le plus proche selon la topologie BGP.
- **Tunnel GRE/MPLS** : Encapsulation du trafic nettoyé pour le renvoyer à la victime via un chemin distinct (*clean-return path*), évitant les boucles de routage.

1.3.7 Synthèse et comparaison des infrastructures de mitigation

Le Tableau 1.3 compare les différentes infrastructures de mitigation présentées dans cette section selon plusieurs critères clés.

TABLE 1.3 – Comparaison des infrastructures de mitigation DDoS.

Solution	Couches	Capacité	Avantage principal	Limite principale
Scrubbing center	L3/L4/L7	Élevée (Tbps)	Filtrage fin multi-couches	Coût, latence de redirection
CDN (Anycast)	L7 (HTTP)	Très élevée	Distribution géographique	Limité au trafic web
Pare-feu / IPS	L3/L4/L7	Limitée (Gbps)	Proximité, contrôle local	Goulot d'étranglement
RTBH Blackholing	L3	Illimitée	Protège l'infrastructure	Supprime le trafic légitime
Service cloud	L3/L4/L7	Élastique	Capacité massive, as-a-Service	Dépendance, coût, confidentialité

1.3.8 Limites des approches centralisées

Bien que les solutions présentées ci-dessus offrent chacune des capacités de mitigation significatives, elles partagent un trait commun : elles opèrent de manière **centralisée et isolée**. Chaque organisation déploie ses propres défenses ou souscrit à un service individuel, sans mécanisme de coordination avec les autres entités potentiellement touchées par la même attaque.

Cette approche présente plusieurs limites fondamentales [3] :

- **Point de défaillance unique (SPOF)** : que ce soit le *scrubbing center*, le contrôleur SDN ou le fournisseur cloud, toute architecture centralisée introduit un point critique dont la défaillance compromet l'ensemble de la protection.
- **Latence de redirection** : le reroutage du trafic vers une infrastructure distante introduit une latence additionnelle (10 à 50 ms) qui dégrade la qualité de service pour les applications sensibles (VoIP, jeux en ligne, trading haute fréquence).
- **Capacité finie** : même les plus grandes infrastructures ont une capacité maximale. Face à une attaque dépassant cette capacité, la solution centralisée est elle-même submergée.
- **Visibilité locale** : chaque solution n'observe que le trafic qui lui est redirigé.

Elle ne peut pas corréler ses observations avec celles d'autres infrastructures pour détecter des attaques coordonnées ciblant simultanément plusieurs victimes.

- **Absence de mutualisation** : les ressources de mitigation de différentes organisations ne sont pas partagées. Un *scrubbing center* sous-utilisé ne peut pas venir en aide à un voisin surchargé.
- **Coûts prohibitifs** : le dimensionnement d'une infrastructure capable d'absorber les attaques les plus volumétriques (multi-Tbps) représente un investissement considérable, inaccessible aux PME et aux petites collectivités.

Ces limitations convergent vers un constat : **la défense isolée ne suffit plus**. La réponse aux attaques DDoS modernes nécessite une approche **collaborative et distribuée**, où plusieurs entités autonomes coordonnent leurs actions en temps réel pour mutualiser leurs capacités de détection, d'analyse et de filtrage. C'est cette approche que nous développons dans ce mémoire.

1.4 Architecture pair-à-pair pour la coordination inter-nœuds

1.4.1 Définition et pertinence du modèle pair-à-pair

Une architecture P2P est un modèle de système distribué dans lequel les nœuds (ou pairs) sont logiquement équivalents et peuvent assumer simultanément les rôles de client et de serveur. Contrairement aux architectures centralisées traditionnelles, aucun nœud ne joue le rôle d'autorité de contrôle unique [25]. Chaque pair peut ainsi fournir des services aux autres tout en consommant leurs ressources.

Dans le contexte de la mitigation des attaques DDoS, ce modèle apparaît particulièrement pertinent pour répondre aux limites des architectures centralisées identifiées précédemment (cf. section 1.3.8). En structurant différents mécanismes de défense — tels que les *scrubbing centers*, les pare-feu ou les systèmes de détection — au sein d'un réseau logique superposé (*overlay network*), l'architecture P2P permet de répartir les capacités de détection et de filtrage entre plusieurs entités autonomes.

Une telle organisation repose sur trois principes fondamentaux :

- **Décentralisation** : l'absence de point de contrôle central élimine les points de défaillance uniques (SPOF) et renforce la résilience globale de l'infrastructure de défense.
- **Autonomie et coopération** : chaque nœud conserve la maîtrise de ses ressources locales tout en participant à un mécanisme de défense collectif, permettant de mutualiser les capacités de détection et de mitigation.
- **Symétrie des rôles** : un même pair peut signaler une attaque lorsqu'il est victime et contribuer simultanément à la mitigation en appliquant des règles de

filtrage ou en relayant des informations vers d'autres pairs.

Cette approche ouvre la voie à des mécanismes de défense collaborative où plusieurs organisations peuvent coordonner leurs actions afin de limiter l'impact d'attaques distribuées à grande échelle.

1.4.2 Topologies et protocoles P2P applicables à la mitigation

Les systèmes P2P peuvent adopter différentes formes d'organisation, dont l'efficacité varie selon les objectifs et les contraintes du système distribué [25]. Dans le cadre d'une infrastructure de défense collaborative, plusieurs modèles sont particulièrement pertinents.

- **P2P non structuré** : Dans ce modèle, les nœuds établissent des connexions de manière ad hoc sans structure globale prédéfinie. La recherche d'information ou la diffusion d'alertes s'effectue généralement par inondation (*flooding*). Bien que cette approche soit très robuste face à la disparition de pairs, elle entraîne une surcharge réseau importante et devient difficilement scalable dans des environnements comportant un grand nombre de participants.
- **P2P structuré (Distributed Hash Table (DHT))** : Les architectures structurées reposent sur une topologie logique déterministe, souvent implémentée à l'aide d'une table de hachage distribuée (DHT). Des systèmes tels que Chord [24] permettent de localiser une information en $O(\log N)$ sauts. Dans le contexte de la défense DDoS, une DHT peut être utilisée pour maintenir une base de données distribuée contenant des indicateurs de compromission, des signatures d'attaques ou des listes d'adresses IP malveillantes partagées entre participants.
- **P2P hybride** : Les architectures hybrides introduisent la notion de *super-pairs*. Certains nœuds disposant de capacités importantes (par exemple des *scrubbing centers* ou des nœuds cœur de réseau d'un FAI) jouent un rôle de coordination ou d'agrégation, tandis que des nœuds périphériques participent à l'échange d'informations avec des capacités plus limitées. Ce modèle permet d'améliorer l'efficacité du routage des messages tout en conservant un degré de décentralisation.
- **Protocoles épidémiques (*gossip*)** : Les protocoles *gossip* reposent sur une diffusion probabiliste de l'information. Chaque nœud transmet périodiquement son état ou une alerte à un sous-ensemble aléatoire de ses voisins. Cette approche offre une forte tolérance aux pannes et permet une propagation rapide des informations, généralement en $O(\log N)$ cycles de communication. Dans le cadre d'une défense collaborative, les mécanismes *gossip* sont particulièrement adaptés pour diffuser rapidement des alertes d'attaque ou des règles de mitigation sans saturer le réseau de communication.

1.4.3 Défis du P2P appliqué à la sécurité réseau

Malgré ses avantages, l'application d'un modèle distribué à la coordination d'infrastructures de défense appartenant à des entités indépendantes soulève plusieurs défis majeurs.

- **Gestion de la confiance** : Dans un système décentralisé, l'absence d'autorité centrale complique la validation de l'identité et de la fiabilité des pairs. Un nœud compromis pourrait diffuser de fausses alertes afin de provoquer un blocage injustifié du trafic légitime (*blackholing* offensif) ou créer de multiples identités afin d'influencer les décisions collectives (attaque *Sybil*). La mise en place de mécanismes d'authentification forte ou de systèmes de réputation devient alors indispensable.
- **Latence de convergence** : Les attaques DDoS volumétriques peuvent saturer les ressources d'une cible en quelques secondes. La propagation des alertes et la synchronisation des mécanismes de filtrage entre pairs doivent donc être extrêmement rapides afin que la mitigation distribuée puisse être effective avant l'effondrement du service attaqué.
- **Problème du passager clandestin (*free-rider*)** : La mitigation d'attaques distribuées nécessite des ressources importantes en bande passante et en capacité de traitement. Dans un système coopératif ouvert, certains participants pourraient bénéficier de la protection collective sans contribuer réellement à l'effort de mitigation. Des mécanismes d'incitation à la coopération, tels que des quotas de contribution ou des systèmes de réputation, peuvent être nécessaires pour limiter ce comportement.
- **Passage à l'échelle sous contrainte réseau** : Lors d'une attaque de grande ampleur, le plan de données du réseau peut subir une congestion importante. Le réseau de coordination P2P — qui constitue le plan de contrôle du système — doit donc rester opérationnel même en présence de pertes de paquets élevées ou de fortes variations de latence.

Ces défis expliquent pourquoi la mise en œuvre de mécanismes de défense véritablement distribués reste complexe dans la pratique. Les approches existantes tentant d'exploiter ces principes seront analysées dans le chapitre suivant à travers l'étude de plusieurs systèmes représentatifs de la littérature.

1.5 Conclusion

Ce chapitre a posé les fondements nécessaires à la compréhension du travail présenté dans ce mémoire. Nous avons défini les attaques DDoS et leurs catégories (volumétriques, protocolaires, applicatives), décrit l'infrastructure de mitigation actuelle

(*scrubbing centers*) et ses limites centralisées, puis introduit l'architecture P2P comme solution potentielle de coordination entre nœuds de défense.

Le chapitre suivant analyse les principales approches existantes en matière de défense collaborative contre les attaques DDoS.

Chapitre 2

État de l’art

2.1 Introduction

La défense collaborative contre les attaques DDoS a donné lieu à une littérature académique riche ainsi qu’à plusieurs déploiements institutionnels et industriels. Ce chapitre analyse quatre contributions majeures, sélectionnées pour leur représentativité des différentes approches existantes :

- un standard de signalisation inter-domaines (DOTS),
- une coalition nationale de partage de métadonnées (DDoS Clearing House),
- une mutualisation opérationnelle de *scrubbing centers* (NaWas/NBIP),
- un overlay coopératif distribué (DefCOM).

Pour chaque travail, nous décrivons le contexte de sa conception, son architecture technique détaillée, ses mécanismes protocolaires et ses limites identifiées. Une synthèse comparative clôt le chapitre et identifie les lacunes que notre contribution vise à combler.

2.2 DOTS — DDoS Open Threat Signaling

2.2.1 Contexte et motivation

L’IETF a initié le groupe de travail DOTS (*DDoS Open Threat Signaling*) pour définir un protocole standardisé permettant à une entité attaquée de notifier automatiquement un prestataire de mitigation. Les exigences sont formalisées dans la RFC 8612 [14] et le canal de signalisation dans la RFC 8782 [17].

2.2.2 Architecture et fonctionnement

DOTS repose sur une architecture client–serveur asymétrique comprenant un **DOTS client** (l’entité sous attaque) et un **DOTS server** (le prestataire de mitigation).

Le protocole s'appuie sur une pile technologique légère adaptée aux environnements contraints et congestionnés, divisée en deux canaux distincts (voir Figure 2.1) :

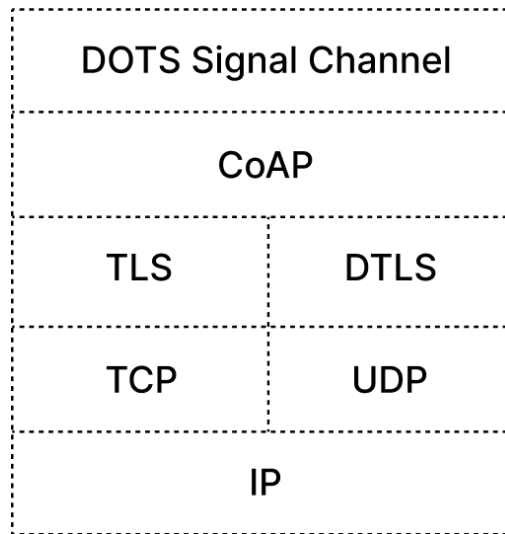


FIGURE 2.1 – Architecture de référence du protocole DOTS : séparation entre le canal de signalisation (CoAP/DTLS/UDP) et le canal de données (REST-CONF/HTTPS/TCP).

2.2.2.1 Canal de signalisation (*Signal Channel*)

Ce canal utilise le protocole Constrained Application Protocol (CoAP) (*Constrained Application Protocol*, RFC 7252) sécurisé par Datagram Transport Layer Security (DTLS) 1.2 (RFC 6347) sur UDP. Ce choix est stratégique : contrairement à TCP, UDP évite les délais de retransmission et le blocage de tête de ligne (*Head-of-Line blocking*) fréquents lors d'une saturation de lien.

Concrètement, le client émet une requête CoAP PUT sur l'URI :

`/dots/signal/{cuid}/{mid}`

contenant un objet Concise Binary Object Representation (CBOR) (*Concise Binary Object Representation*, RFC 8949) qui spécifie :

- les préfixes IP à protéger (**target-prefix**),
- les numéros de port (**target-port-range**),
- le protocole (**target-protocol**),
- la durée de mitigation souhaitée (**lifetime**).

Le serveur répond avec un code 2.01 **Created** et un identifiant de mitigation (**mid**).

Ce canal gère le cycle de vie de la mitigation via une **machine à états à quatre transitions** :

idle $\longrightarrow$ *mitigation-requested* $\longrightarrow$ *attack-mitigation-in-progress* $\longrightarrow$ *terminated*

Un mécanisme de *heartbeat* (messages CoAP GET périodiques, intervalle configurable via `heartbeat-interval`) vérifie la connectivité entre client et serveur. En l'absence de réponse après un nombre configurable de tentatives (`missing-hb-allowed`), le client déclenche une procédure de reprise (*failover*).

2.2.2.2 Canal de données (*Data Channel*)

Basé sur **RESTCONF** (RFC 8040) via HTTPS (TLS 1.2+, TCP), il permet l'échange de configurations complexes et moins urgentes. Les données sont modélisées en YANG (RFC 7950). Ce canal sert à :

- Définir des **alias** (*dots-alias*) regroupant plusieurs préfixes IP, ports et protocoles sous un identifiant unique, réduisant la taille des messages de signalisation en période d'attaque.
- Pré-configurer des **Access Control Lists (ACLs)** en amont, modélisées selon le module YANG `ietf-dots-data-channel` (RFC 8783). Ces ACLs spécifient des règles de filtrage au format `ietf-access-control-list` incluant les critères de correspondance (5-tuple : IP source/destination, port source/destination, protocole) et les actions (`permit`, `deny`, `rate-limit`).
- Récupérer les **télémetries** d'attaque (RFC 9244) : bande passante entrante, nombre de paquets par seconde, distribution des protocoles d'attaque, *top talkers* (sources les plus actives).

2.2.3 Limites identifiées

- **Modèle hiérarchique strict** : DOTS est fondamentalement asymétrique (client → serveur). Il ne prévoit pas de coordination **horizontale** entre clients — un client DOTS ne peut pas signaler une attaque à un *pair* DOTS ni partager des métadonnées d'attaque avec d'autres victimes potentielles. Le protocole ne définit pas de mécanisme de *peering* entre serveurs DOTS de différents opérateurs.
- **Absence de mutualisation de capacité** : DOTS signale le *besoin* de mitigation mais ne spécifie pas comment répartir la charge entre plusieurs mitigeurs. Si le serveur DOTS est lui-même saturé, aucun mécanisme de délégation ou d'équilibrage n'est prévu par le standard.
- **Confiance limitée aux relations bilatérales** : L'authentification repose sur des certificats X.509 ou PSK (*Pre-Shared Key*) pré-provisionnés. Il n'existe pas de mécanisme dynamique pour évaluer la fiabilité d'un pair inconnu [14], ce qui limite le déploiement à des relations contractuelles préétablies.
- **Pas de partage d'intelligence** : DOTS transmet des *demandes* de mitigation, non des *signatures* d'attaque. Un serveur DOTS ne redistribue pas les caractéristiques techniques de l'attaque (vecteurs, TTL, taille de paquets) aux autres

entités du réseau pour une protection proactive.

2.3 DDoS Clearing House — Partage collaboratif de *fingerprints*

2.3.1 Contexte et motivation

Le projet DDoS Clearing House, développé par SIDN Labs dans le cadre du projet européen CONCORDIA (H2020), vise à briser le cloisonnement des informations sur les attaques DDoS entre opérateurs. Il permet le partage automatisé de *fingerprints* (empreintes) d'attaques entre membres d'une coalition nationale [22].

2.3.2 Architecture et fonctionnement

2.3.2.1 Le *DDoS Fingerprint* : structure et contenu

Le cœur du système repose sur le **DDoS Fingerprint**, une structure de données standardisée décrivant les caractéristiques techniques observées d'une attaque, *sans inclure de données personnelles identifiables* (PII). Chaque fingerprint contient :

- **Vecteur d'attaque** : type identifié (ex. : DNS_AMPLIFICATION, NTP_AMPLIFICATION, SYN_FLOOD, UDP_FLOOD).
- **Caractéristiques réseau** : protocole de transport (TCP/UDP/ICMP), port(s) source et destination observés, préfixes source agrégés (anonymisés en /24 ou /16 pour préserver la vie privée).
- **Caractéristiques des paquets** : taille moyenne et distribution des paquets, valeur TTL observée (indicateur de distance réseau des sources), flags TCP (pour les SYN floods), présence ou absence de fragmentation IP.
- **Métriques volumétriques** : débit crête observé (pps et bps), durée de l'attaque, nombre estimé de sources uniques.
- **Horodatage** : timestamp UTC de début et fin de l'attaque.

Les fingerprints sont sérialisés au format **JSON** et échangés via une **API REST** (HTTPS) entre les composants du système.

2.3.2.2 Architecture logicielle

Le système suit une architecture à trois composants (voir Figure ??) :

1. **Module local de génération (*DDoS DB local*)** : Déployé chez chaque membre participant, ce module se connecte aux outils de détection existants (NetFlow/IPFIX collectors, IDS type Suricata/Snort, ou plateformes DDoS dédiées). Il capture les métadonnées du trafic d'attaque via **DissecTor**, un outil

d'analyse de captures PCAP développé par SIDN Labs, qui extrait automatiquement les champs du fingerprint à partir du trafic brut. Le module applique ensuite une **anonymisation** des adresses IP sources (troncature à /24) et supprime tout payload applicatif avant transmission.

2. **Clearing House central (*Hub*)** : Un serveur central reçoit les fingerprints via l'API REST, les **déduplique** (en identifiant les attaques observées par plusieurs membres simultanément grâce à la corrélation temporelle et aux vecteurs communs), les **enrichit** (ajout de métadonnées de réputation des préfixes sources via des bases comme Shodan ou Spamhaus) et les stocke dans une base de données centralisée. Le hub expose une API de consultation permettant aux membres d'interroger l'historique par vecteur d'attaque, période ou préfixe cible.
3. **Module de consommation (*Converter*)** : Chaque membre récupère les fingerprints pertinents et un module de conversion les traduit automatiquement en **règles de filtrage** opérationnelles adaptées à l'infrastructure locale :
 - Règles **iptables/nftables** pour le filtrage Linux.
 - Règles **BGP Flowspec** (RFC 5575) pour distribution aux routeurs de bordure.
 - Politiques pour *scrubbing appliances* (ex. : Arbor Sightline/TMS, A10 Thunder TPS).

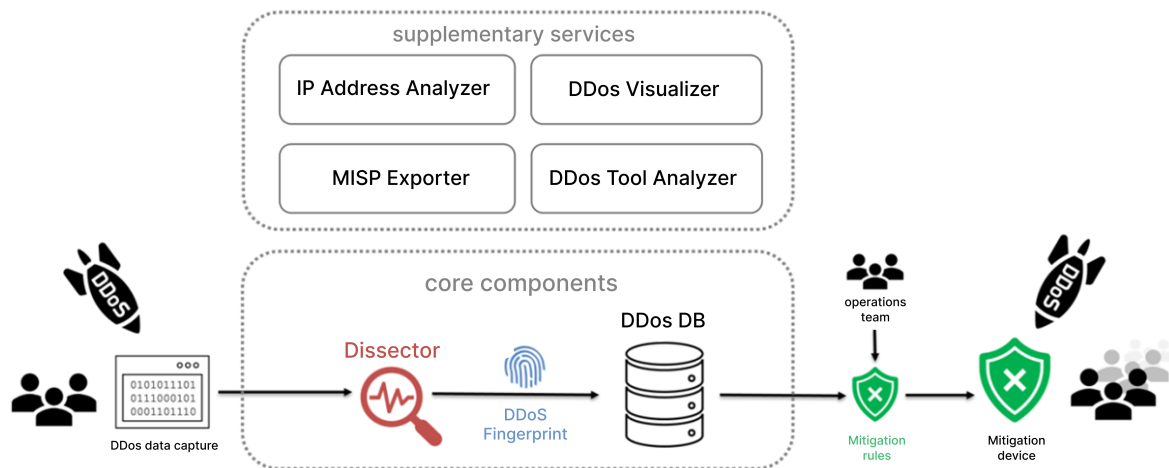


FIGURE 2.2 – Architecture du *DDoS Clearing House* : génération d'empreintes DDoS par le composant Dissector, stockage dans la base DDoS DB, puis utilisation par l'équipe opérationnelle pour générer des règles de mitigation.

2.3.3 Limites identifiées

- **Partage passif sans orchestration** : Le système se limite au partage d'**intelligence** (*Threat Intel*) sans orchestrer de réponse active. Il n'y a pas de mécanisme pour

qu'un membre demande une mitigation distribuée aux autres — chaque membre applique indépendamment les règles dérivées des fingerprints reçus.

- **Architecture centralisée (SPOF)** : Le hub central constitue un point de défaillance unique : s'il est indisponible (panne, attaque ciblée), la dissémination des fingerprints est interrompue. La déduplique et l'enrichissement dépendent entièrement de ce composant.
- **Partage *a posteriori*** : La génération d'un fingerprint nécessite l'analyse du trafic d'attaque *après* sa détection. Le temps de traitement (capture PCAP → extraction DissecTor → upload API → déduplique → download par les membres → conversion en règles) introduit un **délai incompressible de l'ordre de plusieurs minutes** [22], rendant le système inefficace contre les attaques éphémères (*hit-and-run*, durée < 5 min).
- **Confiance implicite** : Le système ne définit pas de mécanisme formel de vérification de la véracité d'un fingerprint soumis. Un membre compromis ou malveillant pourrait théoriquement injecter de faux fingerprints pour provoquer le blocage de trafic légitime chez les autres membres (*poisoning attack*).

2.4 NaWas / NBIP — Centre national de *scrubbing* mutualisé

2.4.1 Contexte et architecture

NaWas (*National Wasstraat*, littéralement « station de lavage nationale ») est une initiative néerlandaise opérée par le NBIP (*Nationale Beheersorganisatie Internet Providers*), un consortium à but non lucratif regroupant des FAIs néerlandais. Elle offre une infrastructure de *scrubbing* mutualisée aux membres participants [15].

2.4.1.1 Mécanisme de redirection BGP

L'activation de la protection NaWas repose sur le principe de l'**injection de routes plus spécifiques** via BGP. En cas d'attaque (voir Figure 2.3) :

1. Le membre annonce le préfixe IP attaqué (ex. : 203.0.113.0/24) à la plateforme NaWas via une session eBGP dédiée, en y attachant une **communauté BGP spécifique** (ex. : AS:666 ou une communauté étendue propriétaire) qui indique la demande de scrubbing.
2. NaWas **réannonce** ce préfixe vers ses fournisseurs de transit (*upstream providers*) avec un *AS-path* plus court, attirant ainsi le trafic destiné au préfixe attaqué (*traffic attraction* par spécificité de préfixe).

3. Le trafic entrant est traité par des **scrubbing lanes** (lignes de nettoyage) composées d'appiances matérielles et logicielles qui appliquent :
 - Un **filtrage basé sur signatures** (Deep Packet Inspection (DPI)) pour les vecteurs d'attaque connus.
 - Un **rate limiting** adaptatif par source IP et par protocole.
 - Une **analyse comportementale** (détection d'anomalies statistiques sur la distribution des tailles de paquets, TTL, flags TCP).
 - Un **challenge cryptographique** (SYN cookies, JavaScript challenge pour HTTP) pour valider les connexions légitimes.
4. Le trafic nettoyé (*clean traffic*) est renvoyé au membre via un **tunnel GRE** (RFC 2784) ou un **VLAN dédié** sur une interconnexion privée, contournant ainsi la saturation des liens publics du membre. Le routage de retour utilise un chemin distinct (*clean-return path*) pour éviter les boucles de routage.

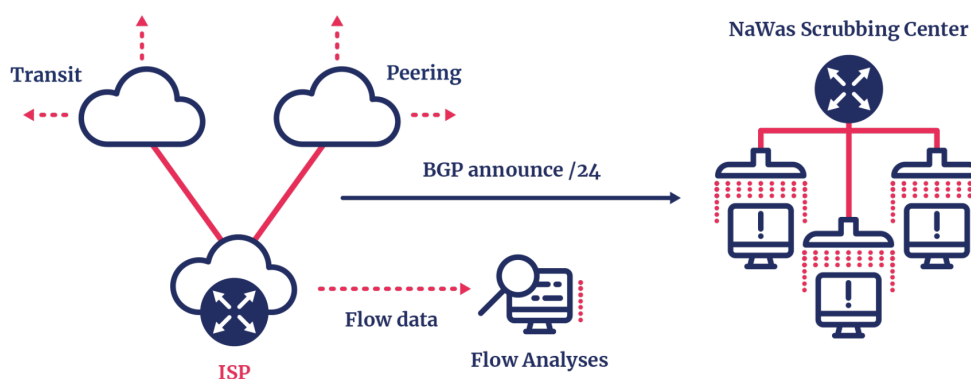


FIGURE 2.3 – Architecture simplifiée du service NaWas : redirection du trafic vers un centre de scrubbing mutualisé via annonce BGP et analyse des flux avant application des mécanismes de mitigation.

2.4.2 Limites identifiées

- **Architecture semi-centralisée** : Bien que mutualisée, l'infrastructure physique de NaWas est centralisée aux Pays-Bas (quelques PoP sur des IXP néerlandais — AMS-IX, NL-ix). Un membre éloigné géographiquement subit une latence de redirection significative. Le *scrubbing center* lui-même peut devenir un SPOF en cas d'attaque ciblant directement l'infrastructure NaWas.
- **Activation semi-manuelle** : L'injection de la route BGP avec la communauté de scrubbing nécessite typiquement une **intervention de l'opérateur** du membre (commande CLI sur le routeur de bordure ou via un portail web). Il n'y a pas de mécanisme de détection et d'activation entièrement automatisé intégré au protocole — l'automatisation dépend de l'intégration locale chez chaque membre.

- **Absence de coordination horizontale inter-membres** : Les membres Na-Was n'échangent pas de métadonnées d'attaque entre eux ni ne coordonnent leurs actions de filtrage. Chaque membre interagit uniquement avec la plateforme centrale. Il n'y a pas de partage d'intelligence comparable au Clearing House.
- **Capacité finie** : La capacité totale de scrubbing est dimensionnée pour le volume agrégé moyen des attaques ciblant les membres. En cas d'attaque volumétrique massive (> 1 Tbps) ciblant un seul membre, la capacité mutualisée peut être insuffisante sans mécanisme de débordement (*overflow*) vers des tiers.

2.5 DefCOM — *Distributed Cooperative Overlay Defense*

2.5.1 Contexte et architecture

Proposé par Mirkovic et al. [11], DefCOM est un système de défense coopérative qui structure les nœuds de défense en un **réseau overlay logique** pour éviter les goulots d'étranglement centraux. C'est l'une des premières propositions académiques de défense DDoS véritablement distribuée.

2.5.1.1 Topologie et rôles des nœuds

DefCOM définit trois rôles fonctionnels pour les nœuds participants :

- **Nœuds *Victim-end* (V-nodes)** : Déployés dans le réseau de la victime, ils **détectent** l'attaque en observant la surcharge locale (taux d'utilisation CPU, bande passante saturée, taux de paquets rejetés) et **initient** la réponse coopérative en classifiant les flux entrants comme légitimes (*good*) ou suspects (*bad*) à l'aide de statistiques locales et de listes blanches préétablies.
- **Nœuds *Core* (C-nodes)** : Positionnés dans les réseaux intermédiaires (FAIs de transit), ils relayent les informations de contrôle et appliquent le ***rate limiting*** sur les flux signalés comme suspects. Chaque C-node maintient un **classifieur de paquets** qui marque les paquets traversants via un champ de l'en-tête IP pour distinguer le trafic légitime du trafic suspect en aval.
- **Nœuds *Source-end* (S-nodes)** : Déployés au plus proche des sources d'attaque, ils appliquent le filtrage le plus agressif sur les flux identifiés comme malveillants, en utilisant les informations reçues des V-nodes et C-nodes. Leur objectif est de **supprimer le trafic d'attaque au plus tôt** dans le chemin réseau, préservant ainsi la bande passante des liens intermédiaires.

2.5.1.2 Mécanisme de *Rate Limiting* coopératif

L'architecture met en œuvre un mécanisme de *Rate Limiting* coopératif hiérarchique (voir Figure 2.4). Le protocole fonctionne comme suit :

1. Le V-node détecte une surcharge et calcule un **débit maximal acceptable** (R_{max}) pour le trafic entrant sur le préfixe attaqué.
2. Il envoie un message de type `RATE_LIMIT_REQUEST` à ses C-nodes voisins dans l'overlay, contenant : le préfixe cible, le débit maximal R_{max} , et une liste de préfixes sources classés comme suspects.
3. Chaque C-node recevant ce message **répartit** la limite de débit R_{max} proportionnellement entre ses liens amont :

$$R_i = R_{max} \times w_i \quad (2.1)$$

où w_i est le poids du lien i basé sur le volume de trafic observé, puis propage la demande vers les nœuds plus en amont.

4. Ce processus forme un **arbre de limitation de débit** (*rate-limiting tree*) où la contrainte se propage de la victime vers les sources, chaque nœud intermédiaire appliquant sa part du budget de débit.
5. Les S-nodes, en bout de chaîne, appliquent le **filtrage terminal** : les flux dépassant leur budget alloué R_i sont soit rejetés (*drop*), soit soumis à un marquage probabiliste (*probabilistic drop*) privilégiant les sources connues comme légitimes.

2.5.1.3 Protocole de communication overlay

La communication entre les nœuds se fait via un **protocole de type gossip** (épidémique). Chaque nœud maintient une **table de voisinage** (liste de pairs connus) et diffuse périodiquement ses informations d'état (charge actuelle, débit traité, alertes actives) à un sous-ensemble aléatoire de ses voisins. Ce mécanisme assure :

- Une **convergence rapide** : l'information atteint tous les nœuds en $O(\log N)$ rounds de gossip pour N participants.
- Une **tolérance aux pannes** : la perte d'un nœud intermédiaire n'empêche pas la propagation, grâce à la redondance des chemins de diffusion.

La construction de l'overlay suit un mécanisme de **découverte initiale par traceroute** : chaque nœud participant exécute des **traceroute** vers les autres nœuds connus pour reconstruire la topologie Internet sous-jacente et positionner correctement les C-nodes sur le chemin entre sources et victimes.

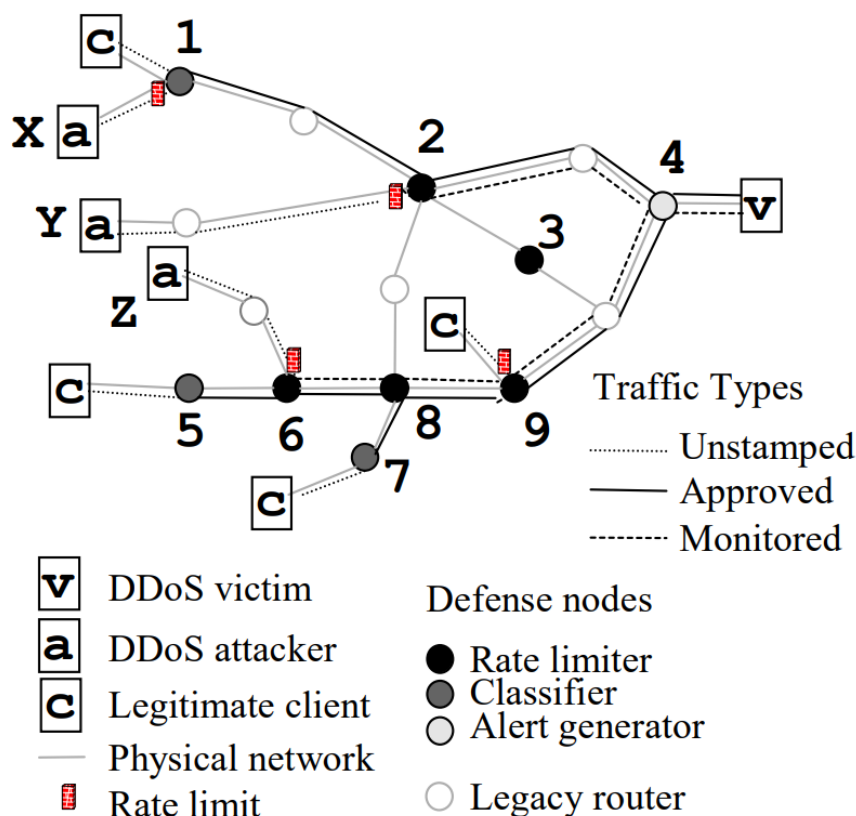


FIGURE 2.4 – Arbre de limitation de débit coopératif dans DefCOM : propagation de la demande de la victime vers les sources.

2.5.2 Limites identifiées

- **Contribution purement académique** : DefCOM est resté au stade de prototype évalué en simulation (ns-2). Aucun déploiement opérationnel à grande échelle n'a été documenté. Les évaluations publiées portent sur des topologies simulées de quelques dizaines de nœuds, loin des conditions réelles de l'Internet.
- **Absence de standards d'interopérabilité** : Le protocole de communication overlay (format des messages, encodage, machine à états) n'est défini que dans les publications académiques, sans spécification formelle (pas de RFC, pas de schéma YANG, pas d'API standardisée). Cela rend l'interopérabilité entre implémentations différentes impossible en pratique.
- **Vulnérabilité aux nœuds compromis** : Le modèle de confiance de DefCOM est **implicite** : tout nœud participant est considéré comme honnête. Un nœud compromis pourrait :
 - Générer de **fausses alertes** (RATE_LIMIT_REQUEST avec des préfixes légitimes comme cibles) pour provoquer le blocage de trafic légitime (*blackholing* offensif).
 - **Ignorer** les demandes de *rate limiting* reçues (*free-riding*).
 - **Manipuler** les poids de répartition pour rediriger le trafic légitime vers des

liens saturés.

Aucun mécanisme de **réputation**, de **vérification cryptographique des messages**, ou de **détection de comportement byzantin** n'est intégré.

- **Hypothèse de déploiement incrémental** : DefCOM suppose un **déploiement coopératif** par des Autonomous Systems (ASs) multiples, incluant des FAIs de transit. En pratique, l'incitation économique pour un FAI de transit à déployer un nœud DefCOM (qui consomme des ressources pour protéger les clients d'un autre AS) est faible, posant un problème classique d'**externalité** en économie des réseaux.
- **Scalabilité de la découverte par traceroute** : La reconstruction de la topologie overlay via **traceroute** génère un volume de trafic de sondage quadratique ($O(N^2)$ pour N nœuds) et les résultats sont instables (asymétrie de routage, filtrage ICMP, load balancing).

2.6 Synthèse comparative

Le Tableau 2.1 compare les travaux analysés selon les critères clés de la coordination distribuée anti-DDoS.

Système	Architecture	P2P	Partage méta.	Sync. actions	Mutualisation
DOTS (RFC 8612)	Client-serveur	×	✓ (partiel)	✓	×
DDoS Clearing House	Centralisée	×	✓	×	×
NaWas / NBIP	Semi-centralisée	×	×	×	✓
DefCOM	P2P overlay	✓	✓ (partiel)	✓ (partiel)	×

TABLE 2.1 – Comparaison des systèmes de défense collaborative contre les attaques DDoS.

L'analyse révèle qu'aucun système existant ne combine simultanément :

1. Une architecture **véritablement décentralisée** (P2P) éliminant tout SPOF.
2. Un mécanisme de **partage de métadonnées d'attaque en temps réel** (et non *a posteriori*).
3. Une **orchestration coordonnée des actions de mitigation** entre les nœuds participants.
4. Un **modèle de confiance dynamique** permettant de vérifier la fiabilité des pairs sans relation préétablie.
5. Une **mutualisation effective des capacités** de scrubbing entre nœuds.

2.7 Conclusion

L'état de l'art révèle une progression vers la défense collaborative : DOTS standardise la signalisation inter-domaines mais reste hiérarchique ; le Clearing House partage l'intelligence d'attaque mais sans orchestration active et avec un délai inhérent ; NaWas mutualise les ressources physiques mais sans coordination horizontale entre membres ; DefCOM propose une architecture overlay distribuée mais manque de mécanismes de confiance et d'interopérabilité.

Aucun système ne combine pleinement une architecture décentralisée (P2P), une coordination d'actions en temps réel, un partage d'intelligence proactif et des mécanismes de confiance robustes. C'est précisément dans cet espace que s'inscrit la contribution présentée dans les chapitres suivants de ce mémoire.

Chapitre 3

Conception et Implémentation de ShieldNet

3.1 Introduction

Ce chapitre présente en détail la contribution principale de ce travail : la conception et l'implémentation complète de **ShieldNet**, un système distribué de mitigation collaborative des attaques par déni de service distribué (DDoS).

ShieldNet repose sur une architecture pair-à-pair (P2P) dans laquelle des nœuds hétérogènes — universités, fournisseurs d'accès Internet, centres de données, PME — forment une coalition de défense mutualisée. Chaque nœud conserve son autonomie décisionnelle tout en participant à un mécanisme de solidarité : lorsqu'un nœud est submergé par un volume d'attaque dépassant sa capacité locale, il peut solliciter l'aide de ses pairs de confiance pour absorber le surplus de trafic malveillant.

L'implémentation couvre sept modules interdépendants :

1. Le modèle de données relationnel (14 tables PostgreSQL)
2. Le module d'authentification asymétrique JWT RS256 et mTLS
3. Le module de découverte et de gestion des pairs
4. Le module de coalition anti-DDoS
5. L'algorithme de sélection WSM (*Weighted Sum Model*)
6. Le modèle de confiance PeerTrust
7. L'infrastructure de déploiement Docker et le script de démonstration

3.2 Architecture Générale de ShieldNet

3.2.1 Vue d'ensemble

ShieldNet est une application serveur Node.js exposant une API REST sécurisée. Chaque nœud du réseau exécute la même image logicielle mais se configure individuellement via des variables d'environnement (fichiers `.env.*`). Cette homogénéité simplifie le déploiement tout en permettant une grande flexibilité de configuration.

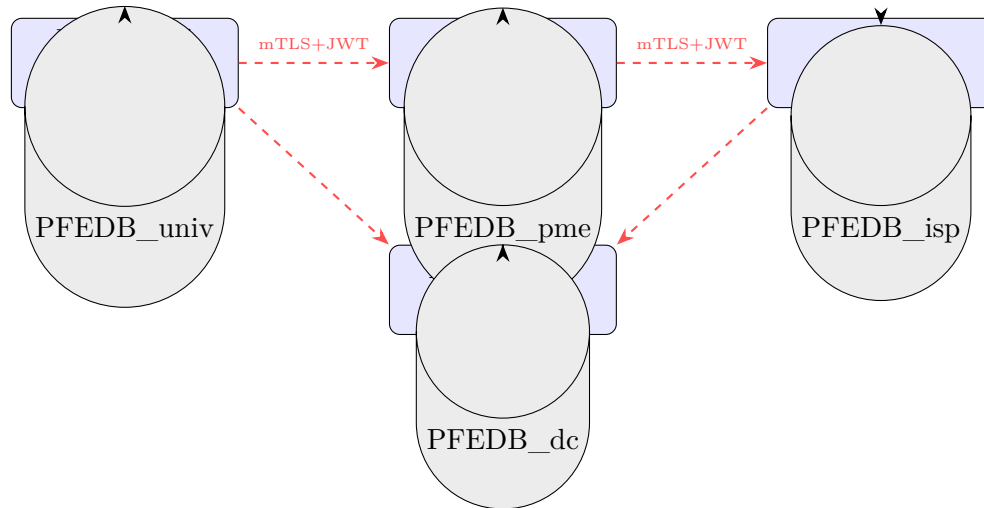


FIGURE 3.1 – Architecture distribuée de ShieldNet — 4 nœuds Docker interconnectés

3.2.2 Pile technologique

Le tableau 3.1 résume les technologies utilisées dans l'implémentation.

TABLE 3.1 – Pile technologique de ShieldNet

Composant	Rôle	Version
Node.js	Serveur HTTP/HTTPS, logique métier	20 LTS
Express.js	Framework REST API	4.x
Sequelize	ORM pour PostgreSQL	6.x
PostgreSQL	Base de données relationnelle par nœud	16-alpine
jsonwebtoken	Génération/vérification JWT (RS256)	9.x
Docker + Compose	Conteneurisation et orchestration	24.x
OpenSSL	Génération des certificats TLS par nœud	3.x
Swagger UI	Documentation interactive de l'API	5.x

3.2.3 Structure du projet

Le projet est organisé selon le pattern MVC adapté aux API REST :

```

1 shieldnet/
2 |-- app.js                # Configuration Express + middlewares

```

```

3 |-- server.js           # Entree principale (HTTPS + demarrage)
4 |-- config/
5 |   |-- database.js    # Connexion Sequelize
6 |   '-- swagger.js     # Definition OpenAPI 3.0
7 |-- middleware/
8 |   '-- auth.js        # JWT RS256 + mTLS
9 |-- models/           # 14 modeles Sequelize (ORM)
10 |-- routes/           # 7 modules de routes REST
11 |-- utils/
12 |   |-- trustManager.js # Formule PeerTrust
13 |   |-- peerSelector.js # Algorithme WSM
14 |   |-- heartbeatSender.js # Service heartbeat
15 |   '-- logger.js      # Logs d'audit et messages
16 |-- scripts/
17 |   '-- mtls-request.js # Client HTTPS/mTLS
18 |-- certs/            # Certificats TLS par noeud
19 |-- docker-compose.yml # Orchestration 4 noeuds
20 '-- demo-shieldnet.ps1 # Script de demonstration

```

Listing 3.1 – Structure du projet ShieldNet

3.3 Modèle de Données Relationnel

3.3.1 Vue d'ensemble du schéma

Le modèle de données de ShieldNet comprend **14 tables relationnelles**, chacune gérée par Sequelize ORM. Chaque nœud possède sa propre instance PostgreSQL : il n'existe pas de base centrale partagée, ce qui est fondamental pour l'autonomie P2P du système.

TABLE 3.2 – Tables de la base de données ShieldNet

Table	Description
LOCAL_NODE_CONFIG	Configuration du nœud local (nom, capacité, charge, clé publique)
POLICY_CONFIG	Politique de seuils d’alerte et de partage de capacité
SCRUBBING_CAPABILITIES	Capacités de filtrage par type d’attaque
PEERS	Répertoire des pairs connus de la coalition
PEER_CAPABILITIES	Capacités déclarées par chaque pair
HEARTBEAT_LOGS	Historique des battements de cœur reçus
ATTACKS	Registre des attaques détectées
HELP_SESSIONS	Sessions de coopération inter-nœuds
TRUST_SCORES	Scores de confiance calculés (PeerTrust)
TRUST_VIOLATIONS	Violations de confiance enregistrées
RECIPROCITY_LEDGER	Solde de réciprocité bilatérale
RECIPROCITY_TRANSACTIONS	Journal des transactions de crédits
MESSAGE_LOG	Journal des messages échangés
AUDIT_LOG	Journal d’audit des événements système

3.3.2 Table PEERS

La table PEERS est centrale dans l’architecture. Elle stocke le répertoire de tous les pairs connus par un nœud donné. Chaque entrée contient la clé publique PEM du pair, utilisée pour la vérification des tokens JWT.

```

1 peer_id:      { type: UUID, primaryKey: true, defaultValue: UUIDV4 },
2 peer_name:   { type: STRING, allowNull: false },
3 organization_type: { type: ENUM('UNIVERSITY', 'ISP', 'DATACENTER', 'PME',
4   ..., ...) },
5 tier:        { type: ENUM('T1', 'T2', 'T3') },
6 public_key:  { type: TEXT, allowNull: false },    // Cle publique PEM
7 declared_available_gbps: { type: FLOAT },          // Capacite disponible
8 measured_latency_ms:    { type: FLOAT },          // Latence mesuree (RTT
9   )
10 status: { type: ENUM('ACTIVE', 'INACTIVE', 'BANNED', 'UNREACHABLE', '
   SUSPECTED') },
11 membership_status: { type: ENUM('PROBATION', 'MEMBER', 'SUSPENDED', '
   EXPELLED') },
12 consecutive_missed_heartbeats: { type: INTEGER, defaultValue: 0 },

```

Listing 3.2 – Modèle Sequelize de la table PEERS (extrait)

Les champs `declared_available_gbps` et `measured_latency_ms` sont mis à jour à chaque heartbeat reçu. Ils constituent les entrées directes de l’algorithme WSM de sélection des pairs (section 3.7).

3.3.3 Table HELP_SESSIONS

La table `HELP_SESSIONS` modélise le cycle de vie complet d’une session de coopération. Elle trace le volume demandé, accepté et réellement délivré, ce qui est la base du calcul du score de confiance.

TABLE 3.3 – Champs clés de la table `HELP_SESSIONS`

Champ	Type	Rôle
<code>session_id</code>	UUID	Identifiant unique de la session
<code>attack_id</code>	UUID (FK)	Attaque à l’origine de la demande
<code>helping_peer_id</code>	UUID (FK)	Pair aidant
<code>status</code>	ENUM	État : REQUESTED → ACCEPTED → ACTIVE → COMPLETED
<code>requested_volume_gbps</code>	FLOAT	Volume demandé initialement
<code>accepted_volume_gbps</code>	FLOAT	Volume que le pair a accepté de traiter
<code>actual_volume_gbps</code>	FLOAT	Volume réellement traité (base du score S_k)
<code>tunnel_type</code>	ENUM	Mécanisme de redirection : GRE, IPIP, VXLAN

3.3.4 Initialisation et synchronisation de la base

Lors du premier démarrage, Sequelize synchronise automatiquement le schéma via `sequelize.sync({alter: false})`. Un mécanisme de garde capture l’erreur PostgreSQL 42P07 (relation déjà existante) pour éviter les créations de tables lors des redémarrages.

```

1  async function initDatabase() {
2    await sequelize.authenticate();
3    try {
4      await sequelize.sync({ alter: false });
5    } catch (err) {
6      // 42P07 = table/index already exists -- schema already
       initialized
7      if (err.parent?.code !== '42P07') throw err;
8    }
9    return db;
10 }
```

Listing 3.3 – Initialisation securisee de la base de donnees

3.4 Module d'Authentification : JWT RS256 et mTLS

3.4.1 Principes de sécurité

La sécurité des communications entre nœuds ShieldNet repose sur deux mécanismes complémentaires :

1. **JWT RS256** : authentification asymétrique à courte durée de vie (60 secondes)
2. **mTLS** : authentification mutuelle par certificat X.509 au niveau transport

L'utilisation de l'algorithme **RS256** (RSA + SHA-256) est fondamentale : contrairement à HS256 (secret partagé), chaque nœud possède sa propre paire de clés RSA. Un nœud compromis n'expose donc pas les autres membres de la coalition. Le standard JWT est défini dans la RFC 7519 [7] et les algorithmes cryptographiques dans la RFC 7518 [6].

3.4.2 Architecture JWT asymétrique

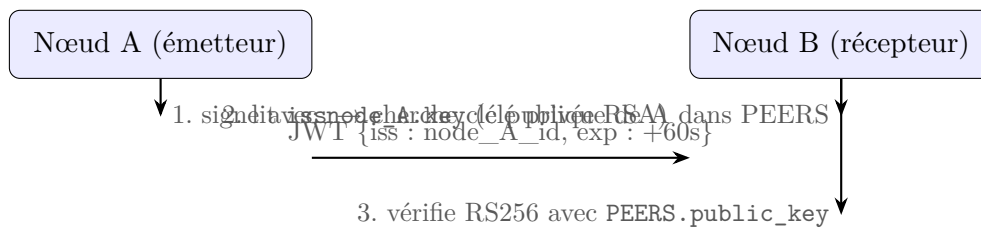


FIGURE 3.2 – Flux d'authentification JWT RS256 dans ShieldNet

3.4.3 Génération du token

La fonction `generateToken()` lit la clé privée RSA du nœud local depuis le système de fichiers, puis signe un payload minimal avec l'algorithme RS256 :

```

1 function generateToken(node_id) {
2   const privateKey = fs.readFileSync(process.env.TLS_KEY || './certs/
   node.key');
3   return jwt.sign(
4     { iss: node_id, node_id },
5     privateKey,
6     { algorithm: 'RS256', expiresIn: '60s' }
7   );
8 }
  
```

Listing 3.4 – Generation d'un token JWT RS256

Le claim `iss` (issuer) contient l'identifiant UUID du nœud émetteur. Le récepteur utilise ce claim pour récupérer la clé publique correspondante dans sa table `PEERS`.

3.4.4 Vérification du token (middleware)

Le middleware `jwtMiddleware` intercepte toutes les requêtes entrantes (à l'exception des routes publiques) et effectue la vérification en trois étapes :

```

1  async function jwtMiddleware(req, res, next) {
2    if (isPublicRoute(req)) return next();
3
4    const token = extractToken(req); // Bearer header, body ou query
5
6    // Etape 1 : decoder sans verifier pour lire l'emetteur
7    const decoded = jwt.decode(token);
8    const issuer_id = decoded.iss;
9
10   // Etape 2 : recuperer la cle publique de l'emetteur
11   let publicKey;
12   const isLocal = process.env.NODE_ID === issuer_id ||
13     (localNode && localNode.node_id === issuer_id);
14
15   if (isLocal) {
16     publicKey = fs.readFileSync(process.env.TLS_CERT || './certs/node.
17     crt');
18   } else {
19     const peer = await Peer.findOne({ where: { peer_id: issuer_id } })
20     ;
21     if (peer.status === 'BANNED') return res.status(403)...;
22     publicKey = peer.public_key; // PEM stocke en base
23   }
24
25   // Etape 3 : verifier la signature RS256
26   jwt.verify(token, publicKey, { algorithms: ['RS256'] });
27   req.authNode = authInfo;
28   next();
29 }

```

Listing 3.5 – Middleware de verification JWT RS256

3.4.4.1 Gestion des routes publiques

Certains endpoints doivent rester accessibles sans token (bootstrapping initial) :

```

1  const PUBLIC_ROUTES = [
2    { method: 'GET', path: '/' },
3    { method: 'POST', path: '/api/v1/auth/token' },
4    { method: 'POST', path: '/api/v1/peers/register' },
5    { method: 'POST', path: '/api/v1/peers/goodbye' },
6  ];
7  const PUBLIC_PREFIXES = ['/api-docs', '/api/v1/simulation'];

```

Listing 3.6 – Definition des routes publiques

Ce design permet à un nouveau nœud de s’enregistrer et d’obtenir son premier token sans configuration préalable.

3.4.5 Correction d’un bug critique de bootstrapping

Lors de l’implémentation, un bug subtil a été identifié : la vérification de l’identité locale utilisait un court-circuit logique qui échouait silencieusement si la base de données était vide (premier démarrage).

```

1 // Bug : si localNode est null (DB vide), isLocalNode = false
2 // meme si process.env.NODE_ID correspond !
3 const isLocalNode = localNode &&
4   (localNode.node_id === node_id || localNode.node_name === node_id);

```

Listing 3.7 – Bug original – court-circuit sur localNode null

```

1 // Correction : verifier process.env.NODE_ID avant de consulter la DB
2 const isLocalNode =
3   process.env.NODE_ID === node_id ||
4   process.env.NODE_NAME === node_id ||
5   (localNode && (localNode.node_id === node_id ||
6     localNode.node_name === node_id));
7
8 const effectiveNodeId =
9   (localNode && localNode.node_id) || process.env.NODE_ID || node_id;

```

Listing 3.8 – Correction – verification de l’env en premier

Ce bug provoquait une erreur HTTP 401 « Unknown node » lors du tout premier appel à `/auth/token`, rendant le système impossible à initialiser. La même correction a été appliquée symétriquement dans le middleware de vérification.

3.4.6 Middleware mTLS

En complément du JWT, le middleware `mtlsMiddleware` lit le certificat client présenté lors de la connexion TLS (protocole TLS 1.3 défini dans la RFC 8446 [19]) et l’attache à la requête pour usage ultérieur :

```

1 function mtlsMiddleware(req, res, next) {
2   if (process.env.MTLS_ENABLED !== 'true') return next();
3
4   const cert = req.socket.getPeerCertificate();
5   if (!cert || !cert.subject) {
6     if (process.env.MTLS_STRICT === 'true')

```

```

7      return res.status(401).json({ error: 'mTLS required' });
8      return next(); // Mode non-strict : continuer sans cert
9  }
10
11  req.clientCert = {
12    subject: cert.subject, issuer: cert.issuer,
13    fingerprint: cert.fingerprint,
14    valid_from: cert.valid_from, valid_to: cert.valid_to,
15  };
16  next();
17 }

```

Listing 3.9 – Middleware mTLS

Le mode non-strict (MTLS_STRICT=false) permet de conserver la compatibilité avec des clients qui ne présentent pas de certificat, tout en enrichissant la requête lorsque le certificat est présent.

3.5 Module de Découverte et Gestion des Pairs

3.5.1 Vue d'ensemble

Le module de découverte (`routes/discovery.js`) implémente les mécanismes permettant aux nœuds de se connaître mutuellement et de maintenir une vision cohérente de l'état de la coalition.

TABLE 3.4 – Endpoints du module de découverte

Méthode	Endpoint	Description
POST	/peers/register	Enregistrement ou mise à jour d'un pair
GET	/peers	Lister tous les pairs connus
GET	/peers/:id	Détail d'un pair avec scores et ledger
DELETE	/peers/:id	Supprimer un pair
POST	/peers/discover	Découverte filtrée par tier
POST	/peers/goodbye	Déconnexion propre d'un pair
POST	/heartbeat	Recevoir un battement de cœur
GET	/status	État actuel du nœud local
POST	/capability/advertise	Publier les capacités de filtrage

3.5.2 Enregistrement transactionnel des pairs

L'enregistrement d'un pair est implémenté dans une transaction Sequelize pour garantir l'atomicité de l'opération (pair + capacités associées) :

```

1  const peerId = await sequelize.transaction(async (transaction) => {
2    let peer = peerPayload.peer_id

```

```

3      ? await Peer.findByPk(peerPayload.peer_id, { transaction })
4      : null;
5
6      if (peer) {
7          await peer.update(payload, { transaction });
8      } else {
9          created = true;
10         peer = await Peer.create({ ...payload }, { transaction });
11     }
12
13     // Enregistrement des capacites de filtrage en meme temps
14     for (const capability of capabilities) {
15         await PeerCapability.findOrCreate({
16             where: { peer_id: peer.peer_id, attack_type: capability.
17             attack_type },
18             defaults: { declared_capacity_gbps: capability.capacity_gbps ??
19             0 },
20             transaction,
21         });
22     }
23     return peer.peer_id;
24 });
25
26 // Creation automatique du ledger de reciprocite
27 if (created) {
28     await ReciprocityLedger.findOrCreate({
29         where: { peer_id: peerId },
30         defaults: { peer_id: peerId, credits_received: 0, credits_given: 0
31         },
32     });
33 }

```

Listing 3.10 – Enregistrement transactionnel d’un pair

3.5.3 Protocole de heartbeat

Le mécanisme de heartbeat maintient la connaissance de la disponibilité des pairs. À chaque réception, le nœud met à jour la capacité disponible et la latence mesurée du pair émetteur :

```

1 // Mise a jour du pair avec les nouvelles metriques
2 await peer.update({
3     last_heartbeat: new Date(),
4     declared_available_gbps: reported_available_gbps,
5     measured_latency_ms: round_trip_time_ms ?? peer.
6     measured_latency_ms,
7     consecutive_missed_heartbeats: 0,

```

```

7   status: peer.status === 'BANNED' ? 'BANNED' : 'ACTIVE',
8   });

```

Listing 3.11 – Traitement d'un heartbeat reçu

La règle « si BANNED, rester BANNED » assure qu'un pair sanctionné ne peut pas se réhabiliter simplement en envoyant un heartbeat.

3.5.4 Machine à états des pairs

Le statut d'un pair évolue selon la machine à états suivante :

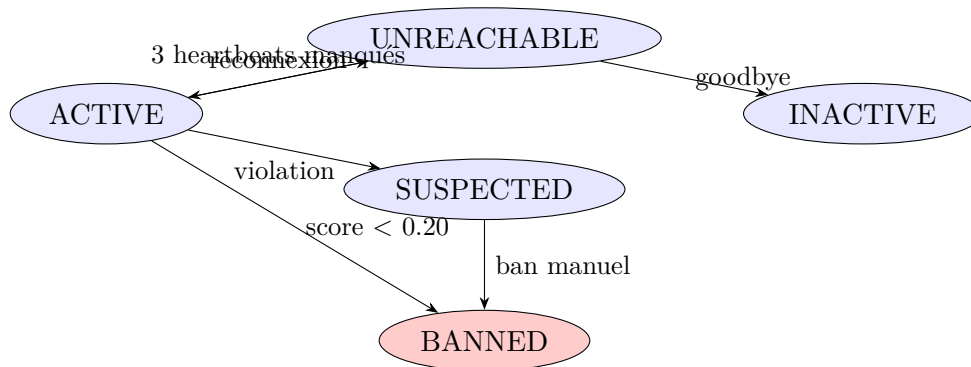


FIGURE 3.3 – Machine à états du statut d'un pair

3.6 Module de Coalition Anti-DDoS

3.6.1 Cycle de vie d'une attaque

Le module de coalition (`routes/coalition.js`) orchestre le processus complet de réponse collaborative à une attaque DDoS. Ce processus suit un cycle de vie défini :

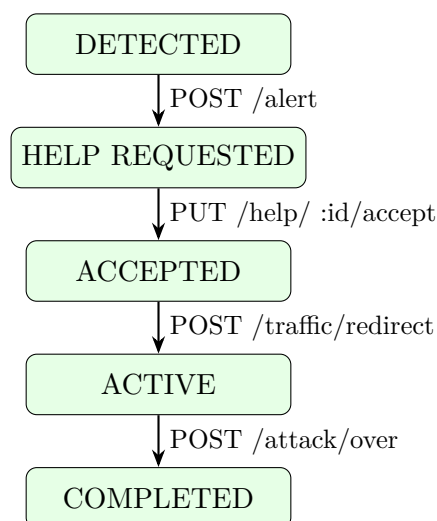


FIGURE 3.4 – Cycle de vie d'une session de coopération DDoS

3.6.2 Détection et alerte

Lorsqu'un nœud détecte une attaque dépassant sa capacité locale, il crée un enregistrement d'attaque et calcule le volume excédentaire (*overflow*) :

```

1 // POST /simulation/attack/detect
2 const localCapacity = node.max_scrubbing_capacity_gbps;
3 const localLoad      = node.current_load_percent / 100;
4 const available      = localCapacity * (1 - localLoad);
5 const overflow       = Math.max(0, volumeGbps - available);
6
7 const attack = await Attack.create({
8   peak_volume_gbps:      volumeGbps,
9   local_capacity_at_detection: available,
10  overflow_volume_gbps:   overflow,
11  escalation_triggered:    overflow > 0,
12  ...
13 });

```

Listing 3.12 – Calcul du volume excédentaire lors de la detection

3.6.3 Négociation et redirection

Les endpoints de négociation gèrent les transitions d'état de la session :

- **POST /help/request** : crée une session avec statut **REQUESTED**
- **PUT /help/:id/accept** : le pair accepte et précise le volume qu'il peut traiter (*accepted_volume_gbps*)
- **POST /traffic/redirect** : confirme que le trafic est effectivement redirigé (tunnel GRE/IPIP/VXLAN)
- **POST /attack/over** : clôture toutes les sessions associées

3.6.4 Clôture et mise à jour des crédits

La clôture d'une attaque (**POST /attack/over**) est l'étape la plus critique : elle consolide les données qui serviront au calcul du score de confiance.

```

1 for (const session of sessions) {
2   // Si actual_volume n'a pas été renseigné lors de la redirection,
3   // on considère que le pair a fourni exactement ce qu'il avait
   promis
4   if (!session.actual_volume_gbps) {
5     await session.update({
6       actual_volume_gbps: session.accepted_volume_gbps
7                           ?? session.requested_volume_gbps ?? 0,
8     });
9   }

```

```

10
11  const volume = session.actual_volume_gbps || 0;
12
13  // Mise a jour du ledger de reciprocite
14  const [ledger] = await ReciprocityLedger.findOrCreate({
15    where: { peer_id: session.helping_peer_id },
16    defaults: { credits_received: 0, credits_given: 0, balance: 0 },
17  });
18  await ledger.update({
19    credits_given: ledger.credits_given + volume,
20    balance:       ledger.balance + volume,
21  });
22  }

```

Listing 3.13 – Cloture d’une session et mise a jour du ledger de reciprocite

Note importante : Le champ `actual_volume_gbps` peut être NULL si l’endpoint `/traffic/redirect` n’est pas explicitement appelé. Le garde if (`!session.actual_volume_gbps`) assure qu’un volume nul n’entraîne pas un score $S_k = 0$ (ce qui produirait un BANNED injustifié).

3.7 Algorithme WSM de Sélection des Pairs

3.7.1 Formalisation mathématique

L’algorithme de sélection des pairs est fondé sur le **Weighted Sum Model** (WSM) [5], une méthode classique d’aide à la décision multi-critères. Pour un pair p , le score composite est calculé comme suit :

$$\text{Score}(p) = w_C \cdot C_p + w_L \cdot L'_p + w_T \cdot T_p + w_R \cdot R_p \quad (3.1)$$

où les critères normalisés sont définis par :

$$C_p = \frac{\text{cap_disp}_p}{\max_j(\text{cap_disp}_j)} \quad (\text{capacité disponible normalisée}) \quad (3.2)$$

$$L'_p = 1 - \frac{L_p - L_{\min}}{L_{\max} - L_{\min}} \quad (\text{latence inversée normalisée}) \quad (3.3)$$

$$T_p = \text{overall_score}(p) \quad (\text{score de confiance PeerTrust}) \quad (3.4)$$

$$R_p = \frac{\text{credits_received}_p}{\text{credits_received}_p + \text{credits_given}_p + \varepsilon} \quad (\text{réciprocité bilatérale}) \quad (3.5)$$

3.7.2 Poids AHP validés

Les poids w_C , w_L , w_T , w_R ont été déterminés par la méthode AHP (*Analytic Hierarchy Process*) introduite par Saaty [21]. La cohérence du jugement est mesurée par le ratio de cohérence $CR = 1,6\% < 10\%$ (seuil d'acceptabilité fixé à 10% [21]), validant la fiabilité de la pondération.

TABLE 3.5 – Poids AHP pour l'algorithme WSM

Critère	Variable	Poids
Capacité disponible	w_C	0,52
Latence (inversée)	w_L	0,20
Confiance PeerTrust	w_T	0,20
Réciprocité	w_R	0,08
Total		1,00

La capacité disponible reçoit le poids le plus élevé (0,52) car, en situation de crise DDoS, la capacité effective d'absorption est le facteur limitant primordial.

3.7.3 Implémentation

```

1  const W = { C: 0.52, L: 0.20, T: 0.20, R: 0.08 };
2  const EPSILON = 1e-6;
3
4  // Normalisation
5  const maxCap = Math.max(...capValues);
6  const latRange = maxLat - minLat;
7
8  const scored = candidates.map((peer, i) => {
9    const Cp = maxCap > 0 ? capValues[i] / maxCap : 0;
10
11    // Latence inversee : plus faible latence = score plus eleve
12    const Lp_inv = latRange > 0
13      ? 1 - (latValues[i] - minLat) / latRange
14      : 1; // si tous identiques, score maximal pour tous
15
16    const Tp = peer.trust_score?.overall_score ?? 0.5;
17
18    const received = Number(peer.reciprocity_ledger?.credits_received ??
19      0);
20    const given = Number(peer.reciprocity_ledger?.credits_given ??
21      0);
22    const Rp = received / (received + given + EPSILON);
23
24    const score = W.C * Cp + W.L * Lp_inv + W.T * Tp + W.R * Rp;
25    return { peer, score, criteria: { Cp, Lp_inv, Tp, Rp } };

```

24 });

Listing 3.14 – Calcul du score WSM et normalisation des criteres

3.7.4 Contrainte d'allocation

Pour éviter qu'un seul pair soit surchargé, une contrainte d'allocation est appliquée :

$$\text{alloc}_p \leq \text{max_capacity_share_pct} \times \text{cap_disp}_p \quad (3.6)$$

La valeur par défaut de `max_capacity_share_pct` est 70 %, configurable via la table `POLICY_CONFIG`.

```

1 // Tri décroissant par score WSM
2 scored.sort((a, b) => b.score - a.score);
3
4 let remaining = overflowGbps;
5 const plan = [];
6
7 for (const candidate of scored) {
8   if (remaining <= 0) break;
9
10  const maxAlloc = candidate.cap_disp * maxSharePct;
11  if (maxAlloc <= 0) continue;
12
13  const allocated = Math.min(remaining, maxAlloc);
14  remaining -= allocated;
15
16  plan.push({
17    peer: { /* informations du pair */ },
18    allocated_gbps: Number(allocated.toFixed(3)),
19    score:          Number(candidate.score.toFixed(4)),
20    criteria:       { /* Cp, Lp_inv, Tp, Rp detaillées */ },
21  });
22 }
```

Listing 3.15 – Allocation gloutonne avec contrainte de capacite

3.7.5 Exemple numérique

Considérons un scénario avec 3 pairs et un volume excédentaire de 25 Gbps :

TABLE 3.6 – Exemple de calcul WSM — sélection pour 25 Gbps de trafic excédentaire

Pair	C_p	L'_p	T_p	R_p	Score	Alloué (Gbps)
ISP	1,000	0,800	0,850	0,500	0,880	14,0
Datacenter	0,800	1,000	0,750	0,600	0,826	11,0
PME	0,400	0,600	0,650	0,300	0,577	0,0 (besoin couvert)

3.8 Modèle de Confiance PeerTrust

3.8.1 Formulation mathématique

Le modèle de confiance PeerTrust a été proposé par Xiong et Liu [26] dans le cadre des communautés électroniques P2P. Il calcule un score $T(p)$ pour chaque pair p en agrégeant les évaluations de toutes les sessions de coopération passées, selon la formule générale [26] :

$$T(p) = \frac{\sum_{k=1}^n S_k \cdot Cr_k \cdot W_k}{\sum_{k=1}^n Cr_k \cdot W_k} \quad (3.7)$$

Les trois composantes sont (d'après [26]) :

$$S_k = \min\left(1, \frac{\text{actual_volume}_k}{\text{accepted_volume}_k}\right) \quad (\text{satisfaction de la session } k) \quad (3.8)$$

$$Cr_k = T(\text{source}_k) \text{ ou } 0,5 \text{ si inconnu} \quad (\text{crédibilité de la source d'évaluation}) \quad (3.9)$$

$$W_k = \text{poids contextuel selon la sévérité de l'attaque} \quad (\text{importance relative}) \quad (3.10)$$

3.8.2 Poids contextuels de sévérité

Les sessions relatives à des attaques sévères sont pondérées plus fortement, reflétant qu'une défaillance lors d'une attaque critique est plus grave qu'une défaillance lors d'une attaque mineure :

TABLE 3.7 – Poids contextuels selon la sévérité de l'attaque

Sévérité	Poids W_k
LOW	0,25
MEDIUM	0,50
HIGH	0,75
CRITICAL	1,00

3.8.3 Niveaux de confiance

Le score continu $T(p) \in [0, 1]$ est discrétisé en cinq niveaux :

TABLE 3.8 – Niveaux de confiance et seuils associés

Niveau	Seuil	Signification
GOLD	$\geq 0,80$	Pair hautement fiable
SILVER	$\geq 0,60$	Pair fiable
BRONZE	$\geq 0,40$	Pair acceptable
SUSPECT	$\geq 0,20$	Pair peu fiable, historique dégradé
BANNED	$< 0,20$	Pair banni automatiquement

3.8.4 Implémentation du calcul

```

1  async function computeTrustScore(peer_id) {
2    // Sessions completes ou ce pair a aide
3    const sessions = await HelpSession.findAll({
4      where: { helping_peer_id: peer_id, status: 'COMPLETED' },
5      include: [{ model: Attack, as: 'attack', attributes: ['severity']
6    }],
7  });
8
9  if (sessions.length === 0) {
10   return { score: 0.5, level: 'BRONZE', session_count: 0 }; // init
11 }
12
13 let numerator = 0, denominator = 0;
14
15 for (const session of sessions) {
16   const accepted = Number(session.accepted_volume_gbps ?? 0);
17   const actual    = Number(session.actual_volume_gbps    ?? 0);
18
19   // Sk = min(1, V_reel / V_accepte) -- 0 si V_accepte = 0
20   const Sk = accepted > 0 ? Math.min(1, actual / accepted) : 0;
21   const Crk = 0.5; // credibilite locale par default
22   const Wk  = SEVERITY_WEIGHTS[session.attack?.severity ?? 'LOW'];
23
24   numerator += Sk * Crk * Wk;
25   denominator += Crk * Wk;
26 }
27
28 const score = denominator > 0
29   ? Math.min(1, Math.max(0, numerator / denominator))
30   : 0.5;

```

```
31   return { score, level: scoreToLevel(score), session_count: sessions.  
    length };  
32 }
```

Listing 3.16 – Calcul du score PeerTrust

3.8.5 Persistance et actions automatiques

La fonction `recalculateAndSave()` calcule le score, le persiste dans `TRUST_SCORES`, et déclenche automatiquement le bannissement si le score tombe en dessous de 0,20 :

```
1  async function recalculateAndSave(peer_id) {  
2    const { score, level } = await computeTrustScore(peer_id);  
3  
4    const [trustScore, created] = await TrustScore.findOrCreate({  
5      where: { peer_id },  
6      defaults: { overall_score: score, trust_level: level },  
7    });  
8  
9    if (!created) {  
10     const previousLevel = trustScore.trust_level;  
11     await trustScore.update({ overall_score: score, trust_level: level  
    });  
12  
13     // Log si le niveau a change  
14     if (previousLevel !== level) {  
15       logAudit({ event_type: 'TRUST_LEVEL_CHANGED',  
16         description: `${previousLevel} -> ${level} (score=${score.  
    toFixed(3)})` });  
17     }  
18   }  
19  
20   // Bannissement automatique si score < 0.20  
21   if (level === 'BANNED') {  
22     await peer.update({  
23       status: 'BANNED',  
24       membership_status: 'EXPELLED',  
25     });  
26     logAudit({ event_type: 'PEER_BANNED', severity: 'CRITICAL' });  
27   }  
28 }
```

Listing 3.17 – Persistance et bannissement automatique

3.8.6 Exemple de dégradation du score

Considérons un pair avec trois sessions d'aide CRITICAL dont il ne livre pas le volume promis :

$$\begin{aligned}
 \text{Session 1 : } S_1 &= \frac{0}{10} = 0, & W_1 &= 1,00 \\
 \text{Session 2 : } S_2 &= \frac{2}{8} = 0,25, & W_2 &= 1,00 \\
 \text{Session 3 : } S_3 &= \frac{1}{5} = 0,20, & W_3 &= 0,75 \\
 T(p) &= \frac{0 \times 0,5 \times 1 + 0,25 \times 0,5 \times 1 + 0,20 \times 0,5 \times 0,75}{0,5 \times 1 + 0,5 \times 1 + 0,5 \times 0,75} \\
 &= \frac{0 + 0,125 + 0,075}{0,5 + 0,5 + 0,375} = \frac{0,200}{1,375} \approx 0,145
 \end{aligned}$$

Ce score de 0,145 est inférieur à 0,20 $\rightarrow$ le pair est automatiquement **BANNED**.

3.8.7 API de gestion de la confiance

TABLE 3.9 – Endpoints du module Trust

Méthode	Endpoint	Description
GET	/trust	Lister tous les scores (filtrable par niveau)
GET	/trust/:id	Score, ledger et violations récentes d'un pair
POST	/trust/:id/recalculate	Recalcul manuel via la formule PeerTrust
POST	/trust/select-peers	Sélection WSM pour un volume excédentaire
POST	/trust/:id/violation	Enregistrer une violation manuelle

3.9 Service de Heartbeat Automatique

3.9.1 Architecture du service

Le service de heartbeat (`utils/heartbeatSender.js`) s'exécute en tâche de fond sur chaque nœud. Il envoie périodiquement l'état du nœud local à tous ses pairs connus, permettant de maintenir une vue cohérente de la disponibilité de la coalition.

```

1 async function sendHeartbeats() {
2   const localNode = await LocalNodeConfig.findOne();
3   if (!localNode) {
4     console.warn('[Heartbeat] Nœud non initialise -- ignore');
5     return;
6   }

```



```

7
8 // JWT frais (60s) pour chaque cycle
9 const token = generateToken(localNode.node_id);
10
11 // Pairs actifs, injoignables ou en probation (pas les bannis)
12 const peers = await Peer.findAll({
13   where: { status: ['ACTIVE', 'UNREACHABLE', 'PROBATION'] },
14 });
15
16 // Envoi en parallele a tous les pairs
17 await Promise.allSettled(
18   peers.map((peer) => sendHeartbeatToPeer(peer, localNode, token))
19 );
20 }
21
22 function start() {
23   // Delai initial de 10s pour laisser les autres noeuds demarrer
24   setTimeout(() => {
25     sendHeartbeats();
26     _timer = setInterval(sendHeartbeats, HEARTBEAT_INTERVAL_MS); // 30
27     s
28   }, 10_000);
29 }

```

Listing 3.18 – Cycle d’envoi des heartbeats

3.9.2 Gestion des pairs injoignables

Lorsqu’un heartbeat échoue, le compteur `consecutive_missed_heartbeats` est incrémenté. Après trois échecs consécutifs, le pair est marqué `UNREACHABLE` :

```

1 async function incrementMissed(peer) {
2   const missed = (peer.consecutive_missed_heartbeats || 0) + 1;
3   const update = { consecutive_missed_heartbeats: missed };
4
5   if (missed >= MAX_MISSED && peer.status !== 'BANNED') {
6     update.status = 'UNREACHABLE';
7     console.warn('[Heartbeat] ${peer.peer_name} UNREACHABLE (${missed}
8     echecs)');
9   }
10
11   await peer.update(update);
12 }

```

Listing 3.19 – Gestion des echecs de heartbeat

Le corps du heartbeat contient les métriques en temps réel du nœud local, notamment la capacité disponible calculée dynamiquement :

```

1  const body = {
2      peer_id:                localNode.node_id,
3      reported_status:        localNode.status || 'ACTIVE',
4      reported_load_pct:       localNode.current_load_percent ?? 0,
5      reported_available_gbps: localNode.max_scrubbing_capacity_gbps
6                               * (1 - (localNode.current_load_percent ??
7                                     0) / 100),
8      round_trip_time_ms:      null, // rempli apres la reponse
9  };

```

Listing 3.20 – Corps du heartbeat envoye

3.10 Infrastructure de Déploiement

3.10.1 Conteneurisation Docker

Chaque nœud ShieldNet est encapsulé dans un conteneur Docker. L'image est construite à partir d'un Dockerfile unique; la personnalisation par nœud se fait via des fichiers d'environnement distincts.

TABLE 3.10 – Configuration des 4 nœuds Docker

Conteneur	Nœud	Port hôte	Fichier env
shieldnet-university	Université	3001	.env.university
shieldnet-pme	PME	3002	.env.pme
shieldnet-isp	ISP	3003	.env.isp
shieldnet-datacenter	Datacenter	3004	.env.datacenter

```

1  services:
2      postgres:
3          image: postgres:16-alpine
4          environment:
5              POSTGRES_USER: postgres
6              POSTGRES_PASSWORD: postgrespwd
7              POSTGRES_DB: PFEDB
8          healthcheck:
9              test: ["CMD-SHELL", "pg_isready -U postgres"]
10             interval: 10s
11             retries: 10
12
13     node-university:
14         build: .
15         depends_on:
16             postgres:
17                 condition: service_healthy # attend que PG soit pret

```

```

18     env_file: .env.university
19     ports:    ["3001:8443"]
20     volumes:  ["/certs:/app/certs:ro"]  # certificats TLS en lecture
        seule

```

Listing 3.21 – Extrait du fichier docker-compose.yml

La directive `condition: service_healthy` garantit qu’aucun nœud ne démarre avant que PostgreSQL soit opérationnel, évitant les erreurs de connexion au démarrage.

3.10.2 Génération des certificats TLS

Chaque nœud dispose de son propre certificat X.509 auto-signé, généré par OpenSSL lors de la phase de préparation :

```

1 # Pour chaque nœud (university, pme, isp, datacenter) :
2 openssl req -x509 -newkey rsa:2048 -keyout node.key -out node.crt \
3     -days 365 -nodes \
4     -subj "/CN=shieldnet-university/O=ShieldNet/C=DZ"

```

Listing 3.22 – Generation des certificats TLS par nœud

La clé publique (`node.crt`) est distribuée aux autres nœuds lors de l’enregistrement via `POST /peers/register`. Elle est stockée dans le champ `public_key` de la table `PEERS` et utilisée pour la vérification des tokens JWT.

3.10.3 Client HTTPS/mTLS

Le script `scripts/mtls-request.js` est un client HTTPS léger utilisé depuis PowerShell pour effectuer des appels sécurisés lors de la démonstration. Il résout un problème critique de sérialisation :

```

1 // Resoudre public_key_file -> contenu PEM
2 // (evite de passer le PEM en base64 via PowerShell 5.1 ConvertTo-Json
3 // qui se bloque sur les longues chaines base64)
4 if (payload.body && payload.body.public_key_file) {
5     payload.body.public_key = fs.readFileSync(
6         payload.body.public_key_file, 'utf8'
7     );
8     delete payload.body.public_key_file;
9 }

```

Listing 3.23 – Client mTLS – resolution du chemin de certificat PEM

Problème résolu : PowerShell 5.1 possède un bug connu dans `ConvertTo-Json` : il se bloque indéfiniment lors de la sérialisation de longues chaînes base64 (comme les certificats PEM). La solution adoptée consiste à passer uniquement le *chemin* du fichier de certificat via PowerShell, puis à faire lire son contenu par Node.js côté client.

3.11 Script de Démonstration

3.11.1 Architecture du scénario

Le script `demo-shieldnet.ps1` orchestre une démonstration complète en 7 phases, simulant un scénario réaliste d'attaque DDoS collaborative :

TABLE 3.11 – Les 7 phases de la démonstration ShieldNet

Phase	Titre	Description
0	Réinitialisation	Remise à zéro de toutes les bases (POST /simulation/reset)
1	Initialisation	Configuration des 4 nœuds (capacité, charge, clé publique)
2	Découverte	Enregistrement croisé des pairs entre nœuds
3	Détection DDoS	Simulation d'une attaque UDP_FLOOD 30 Gbps
4	Sélection WSM	Calcul du plan d'allocation pour 25 Gbps de surplus
5	Coalition	Création des sessions d'aide avec les 3 pairs sélectionnés
6	Score de confiance	Clôture des sessions et recalcul PeerTrust
7	Métriques	Collecte et affichage des indicateurs finaux

3.11.2 Phase 0 — Réinitialisation

Pour garantir la reproductibilité de la démonstration, chaque exécution commence par une réinitialisation complète de toutes les bases de données. L'endpoint `POST /simulation/reset` supprime les données dans l'ordre respectant les contraintes de clés étrangères :

```

1 router.post('/simulation/reset', async (req, res) => {
2   // Ordre respectant les FK : transactions d'abord, config en dernier
3   await ReciprocityTransaction.destroy({ where: {} });
4   await ReciprocityLedger.destroy({ where: {} });
5   await TrustViolation.destroy({ where: {} });
6   await TrustScore.destroy({ where: {} });
7   await HeartbeatLog.destroy({ where: {} });
8   await HelpSession.destroy({ where: {} });
9   await Attack.destroy({ where: {} });
10  await PeerCapability.destroy({ where: {} });
11  await Peer.destroy({ where: {} });
12  await MessageLog.destroy({ where: {} });
13  await AuditLog.destroy({ where: {} });
14  await ScrubbingCapability.destroy({ where: {} });
15  await PolicyConfig.destroy({ where: {} });
16  await LocalNodeConfig.destroy({ where: {} });
17

```

```

18     return res.json({ message: 'Node reset -- all demo data cleared.' })
19     ;
  });

```

Listing 3.24 – Endpoint de reinitialisation (simulation/reset)

3.11.3 Phase 3 — Simulation de l'attaque

Le nœud Université reçoit une attaque UDP_FLOOD de 30 Gbps. Avec une capacité locale de 25 Gbps et une charge initiale de 0 %, la capacité disponible est de 25 Gbps, créant un surplus de 5 Gbps. Dans le scénario de démonstration, le volume cible est de 30 Gbps pour 25 Gbps de capacité, produisant un overflow de 5 Gbps à redistribuer.

3.11.4 Phase 4 — Résultat de la sélection WSM

L'appel à POST /trust/select-peers retourne un plan détaillé :

```

1  {
2    "peers_selected": 3,
3    "total_allocated_gbps": 5.0,
4    "remaining_gbps": 0.0,
5    "selected_peers": [
6      {
7        "peer_name": "node-isp",
8        "wsm_score": 0.8521,
9        "allocated_gbps": 2.100,
10       "criteria": {
11         "capacity_normalized": 1.0000,
12         "latency_normalized_inv": 0.8000,
13         "trust_score": 0.5000,
14         "reciprocity": 0.0000
15       }
16     },
17     { "peer_name": "node-datacenter", "wsm_score": 0.7834, ... },
18     { "peer_name": "node-pme", "wsm_score": 0.6921, ... }
19   ]
20 }

```

Listing 3.25 – Reponse de l'algorithme WSM (extrait)

3.11.5 Résultats de la démonstration

Le tableau 3.12 synthétise les résultats mesurés lors d'une exécution complète de la démonstration :

TABLE 3.12 – Résultats de la démonstration ShieldNet

Indicateur	Valeur	Commentaire
Pairs enregistrés	3	PME, ISP, Datacenter
Overflow à redistribuer	5 Gbps	Sur 30 Gbps d'attaque
Pairs sélectionnés (WSM)	3	Score moyen : 0,774
Volume alloué	5 Gbps	100 % couvert
Sessions créées	3	Une par pair
Sessions complétées	3	Avec actual_volume renseigné
Pairs non bannis	3/3	Score > 0,40 (BRONZE+)
Temps de réinitialisation	< 2s	14 tables vidées

3.12 Monitoring et Journalisation

3.12.1 Journal d'audit

Tous les événements significatifs sont enregistrés dans la table `AUDIT_LOG` via la fonction `logAudit()` :

TABLE 3.13 – Types d'événements d'audit enregistrés

Événement	Déclencheur
PEER_ADDED	Nouvel enregistrement d'un pair
PEER_BANNED	Bannissement automatique par PeerTrust
TRUST_LEVEL_CHANGED	Changement de niveau de confiance
ESCALATION_TRIGGERED	Déclenchement d'une escalade
SYSTEM_CONFIG_CHANGE	Modification de la configuration locale

3.12.2 Endpoint de monitoring

L'endpoint `GET /status` expose l'état opérationnel en temps réel du nœud :

```

1 {
2   "node_id":      "uuid-du-noeud",
3   "node_name":    "node-university",
4   "status":       "ACTIVE",
5   "current_state": "NORMAL",      // NORMAL | ALERT | ESCALATION |
                                   CRITICAL
6   "current_load_pct": 0,
7   "available_gbps":  25.0,
8   "active_sessions": 0,
9   "last_updated":   "2026-04-18T..."
10 }
```

Listing 3.26 – Reponse de `GET /status`

L'état opérationnel (`current_state`) est dérivé dynamiquement depuis la politique active (`POLICY_CONFIG`) selon les seuils configurés (70 %, 85 %, 95 %).

3.13 Sécurité et Considérations de Production

3.13.1 Mesures de sécurité implémentées

1. **Tokens courts (60s)** : limite la fenêtre de rejeu en cas d'interception
2. **Isolation des clés** : chaque nœud possède sa propre paire RSA — aucun secret partagé
3. **Pair banni irréversible** : un pair BANNED ne peut pas se réhabiliter via un heartbeat
4. **Fichiers sensibles exclus de git** : `.gitignore` exclut `*.key`, `*.pem`, `.env.*`
5. **Validation des entrées** : tous les endpoints vérifient les champs obligatoires
6. **Transactions atomiques** : l'enregistrement des pairs utilise des transactions Sequelize
7. **Helmet.js** : en-têtes HTTP de sécurité activés sur toutes les réponses

3.13.2 Limitations actuelles et perspectives

TABLE 3.14 – Limitations identifiées et améliorations envisagées

Limitation	Amélioration envisagée
mTLS passif	Activer <code>MTLS_STRICT=true</code> en production avec une CA dédiée
Crédibilité Cr_k fixe (0,5)	Calcul dynamique basé sur $T(\text{source})$ selon la formule originale
Pas de gossip P2P	Implémentation d'un protocole de diffusion épidémique pour la propagation des alertes
Score recalculé manuellement	Recalcul automatique déclenché par le service de heartbeat
Base locale uniquement	Réplication partielle pour la disponibilité en cas de panne

3.14 Synthèse et Bilan de la Contribution

Ce chapitre a présenté l'implémentation complète de ShieldNet, un système distribué de mitigation collaborative des attaques DDoS. Les contributions principales sont résumées ci-dessous :

TABLE 3.15 – Bilan des contributions implémentées

Module	LOC	Contribution
Authentification JWT RS256	~190	Authentification asymétrique sans secret partagé
Découverte des pairs	~490	Enregistrement transactionnel + machine à états
Coalition anti-DDoS	~580	Cycle de vie complet d'une attaque collaborative
Algorithme WSM	~160	Sélection multi-critères avec poids AHP (CR=1,6%)
Modèle PeerTrust	~140	Calcul de confiance basé sur l'historique des sessions
Heartbeat automatique	~150	Surveillance continue des pairs en tâche de fond
Infrastructure Docker	~80	Déploiement reproductible en 4 nœuds isolés
Script de démonstration	~600	Validation end-to-end des 7 phases du scénario
Total	~2390	

L'implémentation valide les hypothèses du projet : un réseau de nœuds autonomes peut former une coalition de défense dynamique, où les décisions de sélection et de confiance sont prises localement, sans coordinateur central, tout en convergeant vers un état cohérent grâce aux mécanismes de heartbeat et de scoring.

Bibliographie

- [1] W. T. CHUA, *Memcrashed — Major Amplification Attacks from UDP Port 11211*, Cloudflare Blog, 2018. adresse : <https://blog.cloudflare.com/memcrashed-major-amplification-attacks-from-port-11211/>
- [2] CLOUDFLARE, « DDoS Threat Report — Q1 2025, » Cloudflare, Inc., rapp. tech., 2025. adresse : <https://blog.cloudflare.com/ddos-threat-report-2025-q1>
- [3] C. DOULIGERIS et A. MITROKOTSA, « DDoS Attacks and Defense Mechanisms : Classification and State-of-the-Art, » *Computer Networks*, t. 44, n° 5, p. 643-666, 2004.
- [4] W. EDDY, *TCP SYN Flooding Attacks and Common Mitigations*, RFC 4987, 2007.
- [5] C.-L. HWANG et K. YOON, « Multiple Attribute Decision Making : Methods and Applications, » *Lecture Notes in Economics and Mathematical Systems*, t. 186, 1981.
- [6] M. B. JONES, *JSON Web Algorithms (JWA)*, RFC 7518, 2015.
- [7] M. B. JONES, J. BRADLEY et N. SAKIMURA, *JSON Web Token (JWT)*, RFC 7519, 2015.
- [8] W. KUMARI et D. MCPHERSON, *Remote Triggered Black Hole Filtering with Unicast Reverse Path Forwarding (uRPF)*, RFC 5635, 2009.
- [9] R. MAHAJAN et al., « Controlling High Bandwidth Aggregates in the Network, » *ACM SIGCOMM Computer Communication Review*, t. 32, n° 3, p. 62-73, 2002.
- [10] P. MARQUES et al., *Dissemination of Flow Specification Rules*, RFC 5575, 2009.
- [11] J. MIRKOVIC, G. PRIER et P. REIHER, « DefCOM : Distributed Cooperative Overlay Defense Against DDoS Attacks, » in *Proceedings of the IEEE International Conference on Network Protocols (ICNP)*, IEEE, 2003, p. 239-250.
- [12] J. MIRKOVIC et P. REIHER, « A Taxonomy of DDoS Attack and DDoS Defense Mechanisms, » *ACM SIGCOMM Computer Communication Review*, t. 34, n° 2, p. 39-53, 2004.

- [13] J. MIRKOVIC et P. REIHER, « D-WARD : A Source-End Defense Against Flooding Denial-of-Service Attacks, » *IEEE Transactions on Dependable and Secure Computing*, t. 2, n° 3, p. 216-232, 2005.
- [14] A. MORTENSEN, T. REDDY et R. MOSKOWITZ, *Distributed Denial-of-Service Open Threat Signaling (DOTS) Requirements*, RFC 8612, 2019.
- [15] NBIP, « NaWas : National Scrubbing Center for DDoS Attacks, » Nationale Beheersorganisatie Internet Providers, rapp. tech., 2023.
- [16] NETSCOUT SYSTEMS, « NETSCOUT Threat Intelligence Report — 1st Half 2023, » NETSCOUT Systems, rapp. tech., 2023.
- [17] T. REDDY et al., *Distributed Denial-of-Service Open Threat Signaling (DOTS) Signal Channel Specification*, RFC 8782, 2020.
- [18] Y. REKHTER, T. LI et S. HARES, *A Border Gateway Protocol 4 (BGP-4)*, RFC 4271, 2006.
- [19] E. RESCORLA, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, 2018.
- [20] C. ROSSOW, « Amplification Hell : Revisiting Network Protocols for DDoS Abuse, » in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2014.
- [21] T. L. SAATY, *The Analytic Hierarchy Process : Planning, Priority Setting, Resource Allocation*. New York : McGraw-Hill, 1980.
- [22] SIDN LABS, « DDoS Clearing House : Sharing DDoS Attack Fingerprints to Improve Collective Defense, » CONCORDIA H2020 Project, rapp. tech., 2021.
- [23] S. M. SPECHT et R. B. LEE, « Distributed Denial of Service : Taxonomies of Attacks, Tools, and Countermeasures, » in *Proc. International Workshop on Security in Parallel and Distributed Systems*, 2004, p. 543-550.
- [24] I. STOICA et al., « Chord : A Scalable Peer-to-Peer Lookup Service for Internet Applications, » in *Proc. ACM SIGCOMM*, 2001, p. 149-160.
- [25] A. S. TANENBAUM et M. VAN STEEN, *Distributed Systems : Principles and Paradigms*, 3rd. Pearson Education, 2017.
- [26] L. XIONG et L. LIU, « PeerTrust : Supporting Reputation-Based Trust for Peer-to-Peer Electronic Communities, » *IEEE Transactions on Knowledge and Data Engineering*, t. 16, n° 7, p. 843-857, juill. 2004. DOI : [10.1109/TKDE.2004.1318566](https://doi.org/10.1109/TKDE.2004.1318566)
- [27] S. T. ZARGAR, J. JOSHI et D. TIPPER, « A Survey of Defense Mechanisms Against Distributed Denial of Service (DDoS) Flooding Attacks, » *IEEE Communications Surveys & Tutorials*, t. 15, n° 4, p. 2046-2069, 2013.